Igor Kenzo Miyamoto Dias
Marco Antonio Barão Neves

Acessibilidade em emuladores de consoles de videogame por meio de detecção de objetos e reconhecimento de fala

São Paulo - Brasil

2022

Igor Kenzo Miyamoto Dias
Marco Antonio Barão Neves

Acessibilidade em emuladores de consoles de videogame por meio de detecção de objetos e reconhecimento de fala

Monografia apresentada na disciplina Trabalho de Conclusão de Curso, como parte dos requisitos para obtenção do título de Bacharel em Ciência da Computação.

Centro Universitário Senac

Bacharelado em Ciência da Computação

Orientador: Prof. Me. Thyago Quintas

São Paulo - Brasil

2022

Este trabalho é dedicado às crianças adultas que,
quando pequenas, sonharam em se tornar cientistas.

Agradecimentos

Aos nossos familiares e amigos, que incentivaram os participantes do grupo nos momentos difíceis e compreenderam a nossa ausência enquanto nos dedicávamos à realização das atividades do curso.

A todos professores do curso de Ciências da Computação do Centro Universitário Senac Santo Amaro, pelos ensinamentos que nos permitiram apresentar um melhor desempenho no nosso processo de formação acadêmica e profissional ao longo do curso. Em especial para os professores Fábio Abenza, Thyago Quintas, Cristiane Ikenaga, Thiago Claro e Rodrigo Assirati por serem presentemente ativos no desenvolvimento e correção desta dissertação de TCC.

Aos nossos colegas de turma, por compartilharem conosco tantos momentos de descobertas e aprendizado e por todo o companheirismo ao longo deste percurso.

E por fim, aos nossos últimos dois neurônios que sobreviveram.

"Ama-se mais o que se conquista com esforço." Benjamin Disraeli

Resumo

O presente trabalho tem como objetivo apresentar uma prova de conceito de um *software* que, através do uso de inteligência artificial, busca incluir pessoas com deficiências motoras e físicas ao mundo dos jogos digitais, principalmente ao conceito de *retrogaming*, que se refere a *consoles* e jogos antigos que normalmente não possuem tantas funcionalidades com foco em acessibilidade. O produto final será uma camada de entrada que utilizará reconhecimento de fala para permitir que o usuário controle o jogo via comandos de voz. O *software* será responsável por verificar se a ação é válida no estado atual com o uso de detecção de objetos na tela e, caso seja, traduzirá a instrução do jogador para comandos do emulador. O jogo escolhido para o desenvolvimento desse projeto foi o *Pokémon Red*, jogo lançado em fevereiro de 1996 no Japão para o *console Game Boy* e possui algumas opções de acessibilidade implementadas, porém nenhuma delas possibilita uma pessoa com deficiência física ou motora jogar. Com a solução final foi possível demonstrar a viabilidade de jogar o *Pokémon Red* utilizando a fala.

Palavras-chaves: acessibilidade; jogos digitais; *retrogaming*; emulador; detecção de objetos; reconhecimento de fala.

Abstract

The present work aims to present a proof of concept of a software that, through the use of artificial intelligence, seeks to include people with motor and physical disabilities in the world of digital games, especially the concept of retrogaming , which refers to consoles and older games that usually don't have as many accessibility-focused features. The final product will be an input layer that will use speech recognition to allow the user to control the game via voice commands. The software will be responsible for checking if the action is valid in the current state using on-screen object detection and if so, it will translate the player's instruction into emulator commands. The game chosen for the development of this project was Pokemon Red, a game released in February 1996 in Japan for the console Game Boy and has some accessibility options implemented, but none of them allows a person with a physical or motor disability to play. With the final solution it was possible to demonstrate the viability of playing Pokémon Red using speech.

Key-words: acessibility; digital games; retrogaming; emulator; object detection; speech recognition.

Lista de ilustrações

Figura 1 – Mercado Global de Jogos 22
Figura 2 – Arquitetura Von Neumann 24
Figura 3 – Loop de execução em pseudo-código 26
Figura 4 – População residente por tipo de deficiência permanente 27
Figura 5 – Controle adaptável da Microsoft 29
Figura 6 – Jogador utilizando controle adaptável da Microsoft 30
Figura 7 – Definições de IA 30
Figura 8 – Aplicações de Aprendizado de Máquina 31
Figura 9 – Neurônio Artificial 32
Figura 10 – *Perceptron* 33
Figura 11 – Exemplo de ANN 33
Figura 12 – Linha do tempo de marcos da ANN 34
Figura 13 – *Deep NN* 35
Figura 14 – *CNN e suas camadas* 36
Figura 15 – *Exemplo de operação convolucional* 36
Figura 16 – *Exemplo de operação ReLU* 37
Figura 17 – *Exemplo de operação Pooling* 37
Figura 18 – Marcos do ASR 39
Figura 19 – Rede neural da YOLOv1 44
Figura 20 – Arquitetura 49
Figura 21 – Diversas predições no mesmo objeto 53
Figura 22 – Predição antes e depois do NMSBoxes 54
Figura 23 – Contextos escolhidos 54
Figura 24 – Classes anotadas na ferramenta do Roboflow 56
Figura 25 – Fluxograma: *Thread* de estado atual de jogo 57
Figura 26 – Fluxograma: *Thread* de entrada do usuário 58

Lista de tabelas

Tabela 1 – Assertividade no reconhecimento de fala da solução 60

Lista de abreviaturas e siglas

ANN	*Artificial Neural Networks*
API	*Application Programming Interface*
ARPA	*Advanced Research Projects Agency*
ASR	*Automatic Speech Recognition*
CNN	*Convolutional Neural Networks*
CPU	*Central Processor Unit*
CV	*Computer Vision*
DSOD	*Deeply Supervised Object Detectors*
DSSD	*Deconvolutional Single Shot Detector*
FPN	*Feature Pyramid Network*
FPN	*Feature Pyramid Networks*
G-CNN	*Grid Convolutional Neural Network*
GMM	*Gaussian Mixture Models*
HMM	*Hidden Markov Model*
IA	Inteligência Artificial
IBGE	Instituto Brasileiro de Geografia e Estatística
IGDA	*International Game Developers Association*
MLP	*Multi Layer Perceptron*
NN	*Neural Networks*
R-CNN	*Region-based Convolutional Neural Networks*
RAM	*Random Access Memory*
RFCN	*Region-based Fully Convolutional Networks*
ROM	*Read Only Memory*

RPG	*Role Playing Games*
SPP	*Spatial Pyramid Pooling*
SPP	*Spatial Pyramid Pooling Networks*
SSD	*Single-Shot Multibox Detector*
STT	*Speech-to-text*
SUR	*Speech Understanding Research*
VAPA	*Voice-Activated Personal Assistants*
YOLO	*You Only Look Once*

Sumário

1 INTRODUÇÃO . . . 19
1.1 Contexto . . . 19
1.2 Objetivos . . . 20
1.2.1 Objetivos Gerais . . . 20
1.2.2 Objetivos Específicos . . . 20

2 REVISÃO BIBLIOGRÁFICA . . . 21
2.1 Jogos Digitais . . . 21
2.1.1 Evolução da História dos Jogos . . . 21
2.1.2 Mercado . . . 22
2.2 *Retrogaming* . . . 23
2.3 Emulação . . . 23
2.3.1 Emulação x Simulação . . . 24
2.3.2 Arquitetura de um emulador . . . 24
2.4 Acessibilidade para pessoas com necessidades especiais . . . 25
2.4.1 Acessibilidade em jogos . . . 28
2.4.2 Deficiência motora e física . . . 28
2.5 Inteligência Artificial . . . 29
2.5.1 Aprendizado de Máquina . . . 30
2.5.2 *Artificial Neural Networks* . . . 32
2.5.3 *Deep Learning* . . . 34
2.5.4 *Convolutional Neural Networks* . . . 34
2.5.4.1 Camada de Convolução . . . 35
2.5.4.2 Camada de Normalização em Lote . . . 36
2.5.4.3 Camada de ReLU . . . 36
2.5.4.4 Camada de *Pooling* . . . 37
2.5.4.5 Camada totalmente conectada . . . 37
2.5.4.6 Camada de *Soft-max* . . . 38
2.6 Inteligência Artificial para acessibilidade . . . 38
2.6.1 Reconhecimento de Fala . . . 38
2.6.1.1 ASR para acessibilidade . . . 40
2.6.2 Visão computacional . . . 41
2.6.2.1 Detecção de objetos . . . 42
2.6.2.1.1 *Region proposal based framework* . . . 42
2.6.2.1.2 *Regression/classification based framework* . . . 42

2.6.2.2 *You Only Look Once* . . . 43
2.6.2.3 *Darknet* . . . 44
2.6.2.4 OpenCV . . . 45
2.6.2.5 Visão computacional para acessibilidade . . . 45
2.7 Trabalhos relacionados . . . 46

3 DESENVOLVIMENTO . . . 49
3.1 Jogo escolhido . . . 49
3.2 Entrada do usuário . . . 50
3.3 Capturador de tela . . . 51
3.4 Detector de objetos . . . 51
3.4.1 Treinamento . . . 51
3.4.2 Aplicação . . . 53
3.5 Contextos . . . 54
3.5.1 Contexto Genérico . . . 55
3.5.2 Contexto de Batalha . . . 55
3.6 Validação de comandos do usuário . . . 55
3.7 Comandos do emulador . . . 56
3.8 Integrando as partes do sistema . . . 57
3.8.1 *Thread* de estado atual de jogo . . . 57
3.8.2 *Thread* de entrada do usuário . . . 57
3.8.3 Comunicação entre *threads* . . . 57

4 TESTES E RESULTADOS . . . 59
4.1 Resultados . . . 59
4.1.1 Assertividade no reconhecimento de fala . . . 59
4.1.2 Assertividade na detecção do contexto do jogo . . . 60
4.2 Análise de resultados . . . 60

5 CONCLUSÃO . . . 63
5.1 Trabalhos futuros . . . 64

REFERÊNCIAS . . . 65

1 Introdução

Este é um Trabalho de Conclusão de Curso da graduação de Bacharelado em Ciência da Computação do Senac, que utiliza as áreas de inteligência artificial e emulação de hardware para o desenvolvimento de uma interface de controle por fala para um jogo executado em um ambiente de emulador.

1.1 Contexto

Emuladores são meios de recriar o ambiente original em que um determinado software foi projetado para ser executado. Tal característica permite o uso da tecnologia para a preservação digital, um esforço para que as criações digitais continuem acessíveis e utilizáveis, além de combater a obsolescência. Nessa tendência, desenvolvedores começaram a criar emuladores para videogames para conseguir jogar algum produto que parou de ser vendido ou não estava mais disponível (ROSENTHAL, 2015).

A área de jogos digitais teve um crescimento aproximado de 45% no número de usuários no período de 2018 à 2021, aumentando de 2,03 bilhões jogadores para 2,96 bilhões (WIJMAN, 2021). Com necessidade de abranger a maior quantidade de pessoas possível, as empresas investiram na acessibilidade de seus produtos, como foi o caso do "The Last of Us Part II", da Playstation (2020), que virou referência por possuir mais de 60 configurações de acessibilidade, porém nenhuma delas relacionada ao controle por fala.

O reconhecimento de fala, no momento da escrita desse trabalho, é amplamente utilizado através de *"Ok Google"* ou um *"Alexa"*, basta usar esses comandos e o dispositivo escreve uma mensagem ou realiza uma ação. A curiosidade com a "fala" e seu entendimento sempre foram temas de estudo e sua combinação com a área da computação tem como início em 1952, na Bell Labs, quando criaram um sistema que reconhecia números de um dígito, analisando o formante de cada número na fala (JUANG; RABINER, 2004).

Essa combinação é conhecida como ASR - *Automatic speech recognition* (Reconhecimento automático de fala, traduzido do inglês) ou, também, STT - *Speech-to-text* (Fala para texto, tradução livre). Esse avanço tecnológico permitiu não só a automatização de tarefas com a fala, como também a inclusão de pessoas com deficiência que, por meio dos comandos por voz, conseguem utilizar o recurso.

No início da história dos jogos digitais, as desenvolvedoras não tinham a acessibilidade como foco. O interesse pelo assunto cresceu bastante nos tempos atuais devido ao crescimento do mercado, aumento da discussão e regulamentação sobre o tema e pela criação de novas técnicas para incluir pessoas com deficiências (ABLEGAMERS, 2021).

1.2 Objetivos

Para o melhor entendimento dos objetivos deste trabalho, esta seção está dividida em objetivos gerais e específicos, sendo a primeira para definir a questão a ser tratada e a segunda para apresentar os passos para a concepção do resultado final.

1.2.1 Objetivos Gerais

Este trabalho se propõe a implementar uma forma de controle por fala em retro-jogos 2D, do gênero *Role Playing Games* (RPG) por turnos e executado a partir de um emulador com o objetivo de promover maior acessibilidade, especialmente de pessoas com deficiência física ou motora.

1.2.2 Objetivos Específicos

- Criar algoritmo responsável por transformar um comando de fala para um comando do emulador.
- Escolher um software ou biblioteca de reconhecimento de fala;
- Treinar um modelo responsável pelo reconhecimento de objetos do jogo;
- Mapear ações possíveis para os objetos reconhecidos.

2 Revisão Bibliográfica

Neste capítulo são apresentados conceitos fundamentais utilizados no decorrer deste projeto. Os conceitos apresentados são: jogos digitais, acessibilidade, inteligência artificial e inteligência artificial para acessibilidade. Também são detalhados trabalhos relacionados, proporcionando uma ideia do como será atingido o objetivo do projeto.

2.1 Jogos Digitais

Jogos digitais, mais conhecidos como jogos eletrônicos, ou pelo termo em inglês *video games* (ou só *games*), estão presentes desde os anos 50, sendo produzidos por uma equipe pequena, ou até mesmo individualmente.

2.1.1 Evolução da História dos Jogos

O primeiro contato de muitas pessoas com os jogos digitais foi nos *Arcades*, ou Fliperamas como ficaram conhecido no Brasil, pois eram lugares especializados com diversas máquinas, cada uma com um jogo diferente. Os *arcades* também foram responsáveis por popularizar o videogame na década de 80 visto que adolescentes jogavam com seus amigos os principais títulos da época, competindo entre si e abarrotando os estabelecimentos (NOVAK, 2010).

A era dos *arcades* foi muito importante não só enquanto durou, mas também em sua queda, pois deu origem a uma transição muito importante: o início dos *consoles* pessoais. Tais *consoles* eram máquinas especializadas em rodar jogos, que utilizavam a televisão como tela. O primeiro a aparecer foi o *Magnavox Odyssey*, em 1972, que oferecia algumas versões de *Pong*, jogo baseado no *Tennis For Two*, gravadas na memória, e nada mais. Uma das características da primeira geração de videogames era que quando se comprava um jogo, necessariamente se comprava o *console*, pois eram integrados. Ou seja, não era possível para trocar de jogo (PEPE; CONTRIBUTORS, 2022).

A segunda geração foi quem introduziu os cartuchos, também conhecidas como fitas, o que permitiu que jogos fossem lançados por empresas e pessoas diferentes, e vários jogos podiam ser produzidos para o mesmo console. Um ícone dessa geração foi o Atari 2600 que se popularizou pelos clássicos como o *Pitfall* e *Space Invaders*(PEPE; CONTRIBUTORS, 2022).

A partir da evolução dos jogos em cartucho, empresas como a Nintendo ficaram conhecidas, tanto por fazer novos *consoles*, como por criar novos jogos. Ao mesmo tempo, o mercado de computadores pessoais começou a crescer, e na mesma tendência, jogos

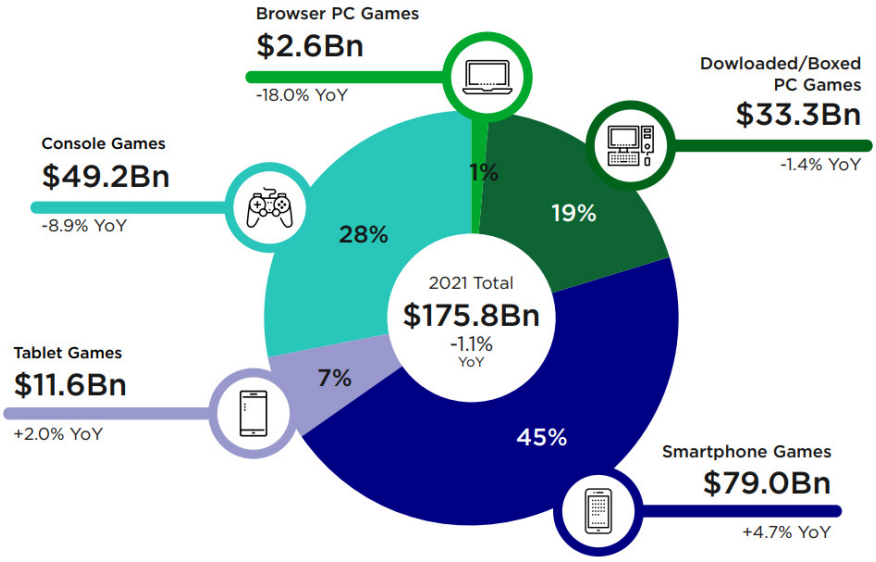
voltados aos computadores começaram a ser desenvolvidos. Com o passar do tempo, avanços tecnológicos permitiram novos tipos de *consoles*, como os portáteis, que funcionam a bateria e possuem tela integrada (PEPE; CONTRIBUTORS, 2022).

2.1.2 Mercado

Com o crescente interesse nesta área, novas empresas foram fundadas e tanto suas equipes de desenvolvimento, quanto seus investimentos aumentaram. Segundo Boltjes (2018), sua equipe alocou mais de 150 pessoas para trabalhar no jogo Horizon Zero Down, lançado em 2017, da Guerrilla Games, e também como indica a matéria do site Push Square, escrito por Barker (2017), que, no mesmo jogo, foram investidos 45 milhões de euros, aproximadamente 54 milhões de dólares, 1 euro representa 1,1995 dólar segundo a fonte BCB (2022), ou 178 milhões de reais, 1 dólar representa 3,3074 reais segundo a fonte BCB (2022).

Em 2020, os games faziam parte da vida de cerca de 40% da população mundial, ou seja, 3,1 bilhão de pessoas jogam algum tipo de videogame (INTELLIGENCE, 2022). Segundo Newzoo Global Games Market Report, o mercado de jogos, em 2021, movimentou aproximadamente 175.8 bilhões de dólares, aproximadamente 980 bilhões de reais, 1 dólar representando 5,5799 reais segundo BCB (2022).

Figura 1 – Mercado Global de Jogos



Fonte: (WIJMAN, 2021)

A Figura 1 mostra algumas plataformas que a área de jogos possui, como também sua receita gerada.

Mesmo que os novos jogos tenham investimentos milionários, videogames mais antigos não perderam sua credibilidade. Pelo contrário, existe um crescente interesse nos

jogos mais antigos (NEWMAN, 2004).

2.2 *Retrogaming*

Retrogaming é um termo em inglês que se refere ao ato de jogar ou colecionar consoles, jogos ou até computadores antigos (ou uma versão mais antiga) do que se encontra presente no mercado. Geralmente, é baseado em sistemas que estão obsoletos ou descontinuados e atua como meio de preservação da história ou pela nostalgia que tal tecnologia pode remeter. (HEINEMAN, 2014)

De acordo com Heineman (2014), para atingir o objetivo, existem alguns métodos que são usados:

- *Vintage*: Pretende preservar o hardware original, então o indivíduo coleciona desde o console, seja ele uma máquina de fliperama ou um portátil, até os cartuchos e discos originais.
- *Emulation*: A emulação de jogos antigos em novo hardware. Utilizando a ROM original, mas não o hardware original.
- *Ported*: Rodar jogos antigos em novos sistemas, mas difere da emulação por que não é usada a ROM original, e sim, a empresa reescreve o jogo para o novo sistema.

Dito isso, achar no mercado tal equipamento para colecionar ou depender da publicadora para reescrever o código de determinado jogo não tem tanta relevância para esse trabalho, pois foge da área da computação. Então a emulação será o foco do próximo tópico.

2.3 Emulação

Emuladores são meios de recriar o ambiente original no qual determinado *software* foi projetado para ser executado (ROSENTHAL, 2015). A emulação de plataformas antigas é amplamente reconhecido como uma estratégia promissora para a preservação digital (TAKHTEYEV; DUPONT, 2013). Ela prove a compatibilidade que falta na migração de *hardware/software*, que acontece cada vez mais nos tempos atuais (ROSENTHAL, 2015).

Essa definição veio junto com a criação do termo para a computação, em 1963, na IBM durante o desenvolvimento da linha de produtos IBM *System*/360 usando uma combinação de *software*, *microcode* e *hardware*. Com essa fusão, descobriram que a execução dos programas escritos para versões anteriores do computador melhoram drasticamente em comparação com apenas simular os comandos antigos usando as instruções novas(USSELMAN, 1995).

2.3.1 Emulação x Simulação

Um emulador é, em alguns aspectos, parecido com um simulador e a maneira que eles divergem é que a simulação imita o comportamento de um sistema, mas sem saber ou reproduzir seu funcionamento interno. Já um emulador tenta reconstruir um sistema a partir do funcionamento do mesmo, de tal forma que, o resultado final seja muito semelhante ao original (BARRIO; FERNÁNDEZ; UPC, 2001).

Gutiérrez (2010) deu um exemplo de reproduzir o funcionamento de uma calculadora HP antiga, para tal, existem duas possibilidades:

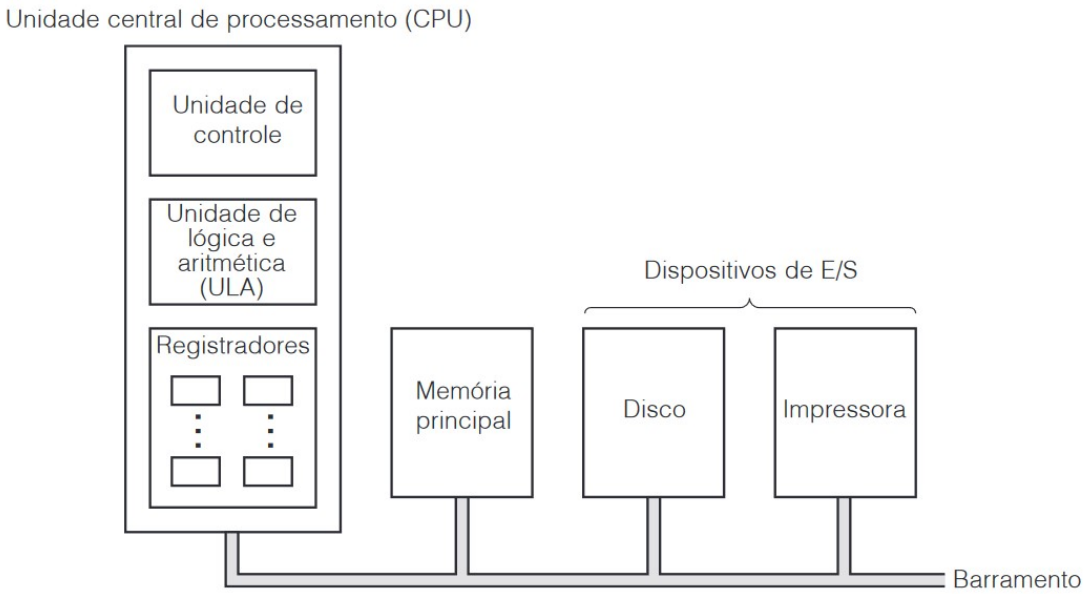
1. Escrever um novo programa que desenha a tela e o teclado da calculadora, que quando usado, o programa faz o que a calculadora antiga fazia. Esse é um simulador.

2. Conseguir o *firmware* (semelhante a uma ROM) original da calculadora e escrever o programa que carrega e interpreta esse *firmware* da mesma maneira que o microprocessador da calculadora faria. Esse é um emulador.

2.3.2 Arquitetura de um emulador

Como cada emulador tenta reproduzir um tipo de sistema diferente, existem diversos emuladores com diversas arquiteturas. Computadores atuais são baseados na arquitetura de Von Neumann, onde os módulos são conectados entre si por um barramento (BARRIO; FERNÁNDEZ; UPC, 2001). A Figura 2 ilustra essa arquitetura.

Figura 2 – Arquitetura Von Neumann



Fonte: Tanenbaum (2013)

A CPU - *Central Processor Unit* (Unidade Central de Processamento, traduzido do inglês) é o módulo que controla o sistema. Ela se comunica com os outros dispositivos pelos barramentos, um grupo de trilhas eletrônicas que transmitem os sinais que os componentes geram para a CPU (TANENBAUM, 2013).

A memória é o módulo que serve como armazenamento de dados primário. Os tipos básicos dessa memória são a RAM - *Random access memory* (Memória de acesso randômico, tradução livre) e a ROM - *Read only memory* (Memória de somente leitura, tradução livre).

A RAM é uma memória volátil, ou seja, ela guarda dados enquanto possui energia, todos os dados são perdidos quando o dispositivo é desligado, sua característica é que pode gravar valores e alterá-los quando necessário (TANENBAUM, 2013).

A ROM é uma memória não volátil, que mantém seu valor mesmo sem energia, e não pode ser sobrescrita. É nesse módulo que ficam o código do programa e os dados que serão executados pela CPU (TANENBAUM, 2013).

Os dispositivos de I/O - Input/Output (Entrada/Saída, tradução do inglês) podem servir para diversos propósitos, como: uma tela, para mostrar algo visualmente, mouse, teclado, controle, para a entrada de dados ou comandos ou outro equipamento que pode ser controlado pela CPU. Resumindo, são os responsáveis por transmitir informações para dentro e para fora do computador(TANENBAUM, 2013).

A CPU lê as instruções, chamadas de opcodes (*operation codes*), da memória, realiza cálculos e retorna o resultado. Também pode realizar interrupções, ou seja, parar de ler instruções para receber um dado de um dispositivo de entrada, como por exemplo: um número do teclado. A CPU também utiliza os dispositivos de saída para enviar informações ao usuário(BARRIO; FERNÁNDEZ; UPC, 2001).

Assim como a CPU é a parte central do computador, na emulação, emular esse processo também vai ser a parte, ou componente central do emulador(BARRIO; FERNÁNDEZ; UPC, 2001).

Segundo Barrio, Fernández e Upc (2001), a execução principal do emulador é seu *loop*, na qual a CPU está sempre fazendo uma busca, decodificação e execução das instruções. Como ela é a parte central, a emulação dos outros módulos seguem a emulação da CPU.

A Figura 3 mostra um exemplo do *loop* principal, em pseudo-código:

2.4 Acessibilidade para pessoas com necessidades especiais

A acessibilidade é a forma que existe para remover obstáculos e proporcionar a igualdade de oportunidades em espaços físicos e digitais para pessoas com necessidades

Figura 3 – Loop de execução em pseudo-código

```
enquanto emular {
    executarCPU(ciclos);
    gerarInterrupção();
    emularGráficos();
    emularSom();
    //emular outros módulos
    sincronizar();
}
```

Fonte: Adaptado de Barrio, Fernández e Upc (2001)

especiais. Segundo a definição encontrada na lei número 10.098, de 19 de dezembro de 2000, acessibilidade é "a possibilidade e condição de alcance para utilização, com segurança e autonomia, de espaços, mobiliários, equipamentos urbanos, edificações, transportes, informação e comunicação, inclusive seus sistemas e tecnologias".

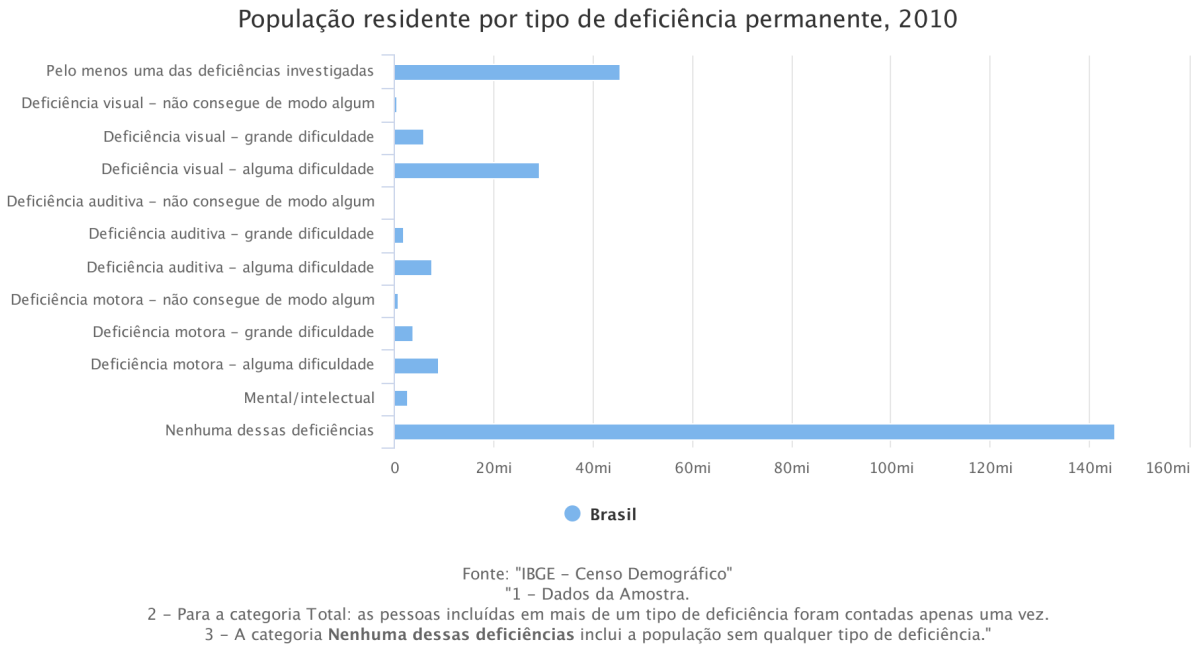
Sassaki (2009) categoriza a acessibilidade na seguinte composição:

- Acessibilidade arquitetônica que combate os entraves encontrados pelas pessoas com necessidades especiais no espaço físico;
- Acessibilidade atitudinal que está relacionado ao atendimento adequado destas pessoas;
- Acessibilidade instrumental diz respeito ao relacionamento e comunicação;
- Acessibilidade programática que busca remover as barreiras burocráticas de normas e regulamentos;
- Acessibilidade no meio digital que visa a melhoria da usabilidade de pessoas com necessidades especiais no mundo digital.

Quando se aborda a questão de representatividade constata-se que aproximadamente 24% da população brasileira possui ao menos um tipo de deficiência, esse número se deve a abrangência da pesquisa, que considera o uso de óculos como parte da deficiência visual parcial e a cegueira como parte deficiência visual total. Os casos são divididos em 18,75% pessoas com deficiência visual, 5% pessoas com deficiência auditiva, 7% pessoas com deficiência motora e 1,36% pessoas com deficiência mental ou intelectual como mostra a

Figura 4 (IBGE, 2010). Quantidade significativa de pessoas que muitas vezes não recebem a atenção necessária e não têm às mesmas condições que as pessoas que não possuem nenhum tipo de deficiência.

Figura 4 – População residente por tipo de deficiência permanente



Com o advento tecnológico, cada vez mais os processos e recursos estão sendo incorporados no meio digital e junto desse movimento não é possível deixar de lado à acessibilidade, pois à tecnologia possui um grande potencial de resolver problemas, garantir acesso à informação, gerar entretenimento, incluir, entre outros (RIBEIRO, 2011). Entretanto, se for mal implementada também tem o poder de excluir esse grupos (CHEIRAN JEAN FELIPE PATIKOWSKI E PIMENTA, 2011).

Segundo Torres e Mazzoni (2004) a acessibilidade no espaço digital é a possibilidade de consumir informações, fazer uso de sistemas e participar de processos de forma autônoma, independentemente de características orgânicas. Para que este espaço se torne mais acessível, os autores também recomendam a disponibilização do conteúdo em mais de uma formato de consumo, por exemplo, ao fazer uso de uma imagem indicam que exista um texto que descreva o significado e contexto desta imagem fazendo com que o conteúdo possa ser consumido por pessoas com deficiência visual através dos seus leitores de tela.

Existem instituições que aplicam esforços para criar diretrizes que tornem o espaço digital mais acessível, como por exemplo, a W3C (Consórcio de rede mundial) que foca principalmente na web disponibilizando especificações e guias de como utilizar componentes de acessibilidade nesse ambiente.

2.4.1 Acessibilidade em jogos

O desafio de incluir pessoas com necessidades especiais se estende também para os videogames. A IGDA (2004) (Associação internacional de desenvolvedores de jogos) define acessibilidade para jogos digitais como a possibilidade de jogar mesmo sob limitações.

No início da história dos jogos eletrônicos, poucas iniciativas e funcionalidades eram desenvolvidas para fazer os jogos acessíveis, mesmo existindo motivos que justificassem a implementação de tais recursos (IGDA, 2004), grandes e pequenas desenvolvedoras da época não deram a devida atenção para o público que possui necessidades especiais (ABLEGAMERS, 2021), gerando uma grande defasagem de conteúdo da época para esses usuários.

Nas últimas duas décadas, houve um aumento expressivo no número de iniciativas criadas para aprimorar a acessibilidade nos jogos, como representante maior desse interesse tem-se o IGDA *Game Accessibility Special Interest Group*, criado em 2003 por Thomas Westin e atualmente liderado por ele, Chad Philip Johnson, Dan Fischbach e Ian Hamilton (IGDA, 2022). Universidades e instituições de pesquisas também demonstraram interesse nessa área de pesquisa trazendo soluções e ferramentas que tornam os jogos cada vez mais viáveis para todos.

Desenvolvedoras passaram a inovar lançando jogos com apenas um controle (one switch games) que focam em simplificar a jogabilidade para pessoas com deficiência motora e cognitiva (SWITCH, 2022), áudio jogos que trazem imersão e diversão para pessoas com deficiência visual (GARCIA; NERIS, 2013) e um grande número de funcionalidades com o foco em acessibilidade (PLAYSTATION, 2020).

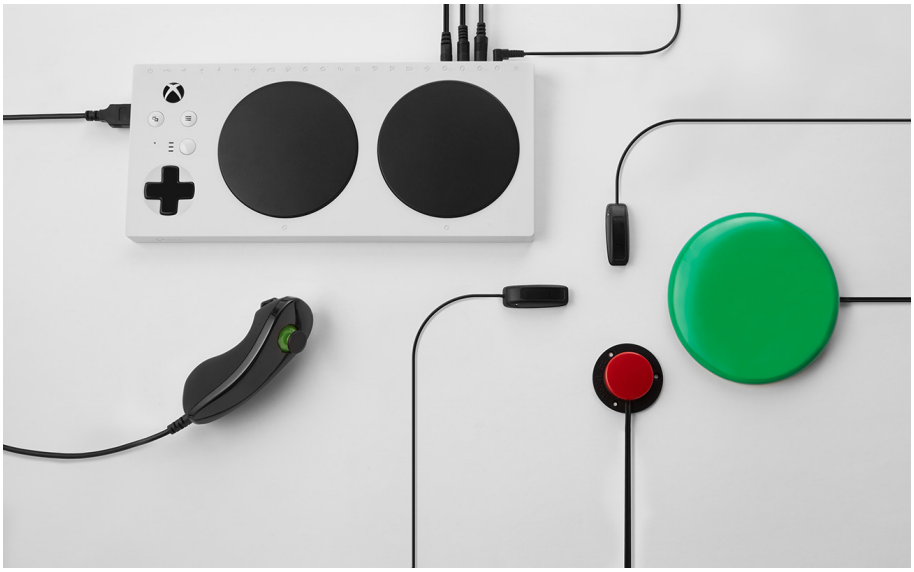
As grandes empresas responsáveis pelos *consoles* também não estão paradas e buscam deixar seus equipamentos acessíveis, como por exemplo, a criação do controle adaptável da Figura 5 pela Microsoft que possibilita a customização de suas entradas para atender a necessidade específica de cada usuário (MICROSOFT, 2018).

2.4.2 Deficiência motora e física

As deficiências física e motora atingem os usuários de jogos digitais pois são condições que dificultam ou impossibilitam as funções dos membros do portador (UFJF, 2022). A seguir alguns tipos desta deficiência são apresentadas (WEBAIM, 2020).

- Tetraplegia - perda total das funções motoras dos membros inferiores e superiores;
- Paralisia cerebral - lesão no cérebro que diminui o controle muscular;
- Artrite - inflamação das articulações que gera deformidade e dificuldade no movimento das mãos;

Figura 5 – Controle adaptável da Microsoft



Fonte: Microsoft (2018)

- Mal de Parkinson e Tremor essencial - desordens do sistema nervoso central que provocam tremores incontroláveis.

Como resultado deste tipo de deficiência, a pessoa portadora pode ter dificuldades para realizar ações através de controles convencionais criando a necessidade de utilizar tecnologias de assistência, controles adaptados como encontrado na Figura 6 ou ferramentas de reconhecimento de fala que possibilitem a interação necessária para jogar, mas mesmo assim sofrem com jogos que exigem precisão nas ações (WEBAIM, 2020).

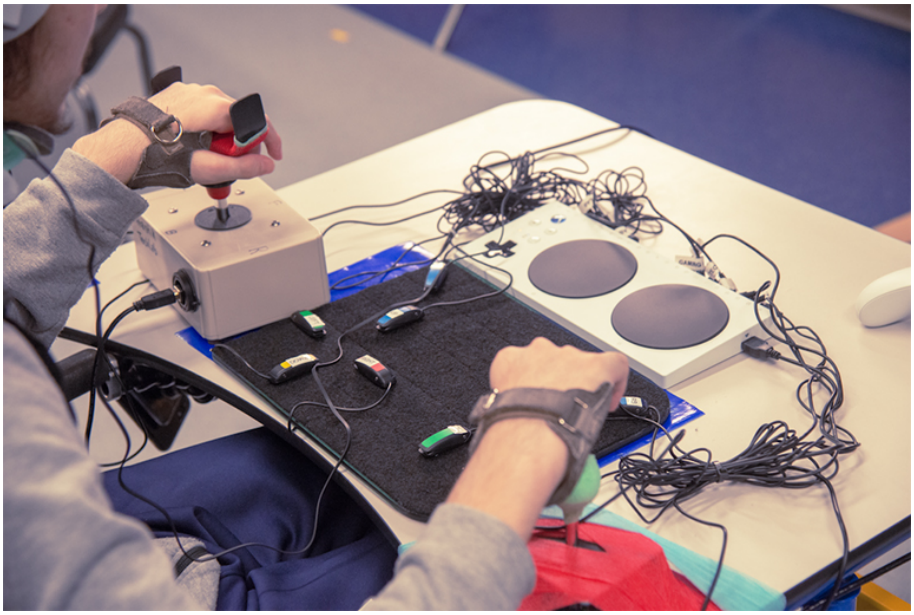
2.5 Inteligência Artificial

Norvig e Russell (2014) mostra diferentes definições de Inteligência Artificial (IA) que surgiram de diferentes estudos que usavam o ser humano como parâmetro e separaram em duas categorias: o que se inspirava no pensamento e outro no comportamento. Dentro dessas categorias, se separaram em duas modalidades de desempenho: o humano, que se assemelha ao desempenho humano, e a outra que chamaram de racionalidade, se o sistema "faz a coisa certa" dado o que ele sabe.

A Figura 7 mostra essa definições:

Não cabe a este trabalho comparar tais definições, visto que será utilizado um subconjunto da IA que é o aprendizado de máquina.

Figura 6 – Jogador utilizando controle adaptável da Microsoft



Fonte: Jung (2018)

Figura 7 – Definições de IA

Pensando como um humano	**Pensando racionalmente**
"O novo e interessante esforço para fazer os computadores pensarem (...) *máquinas com mentes*, no sentido total e literal." (Haugeland, 1985) "[Automatização de] atividades que associamos ao pensamento humano, atividades como a tomada de decisões, a resolução de problemas, o aprendizado..." (Bellman, 1978)	"O estudo das faculdades mentais pelo uso de modelos computacionais." (Charniak e McDermott, 1985) "O estudo das computações que tornam possível perceber, raciocinar e agir." (Winston, 1992)
Agindo como seres humanos	**Agindo racionalmente**
"A arte de criar máquinas que executam funções que exigem inteligência quando executadas por pessoas." (Kurzweil, 1990) "O estudo de como os computadores podem fazer tarefas que hoje são melhor desempenhadas pelas pessoas." (Rich and Knight, 1991)	"Inteligência Computacional é o estudo do projeto de agentes inteligentes." (Poole *et al.*, 1998) "AI... está relacionada a um desempenho inteligente de artefatos." (Nilsson, 1998)

Fonte:Norvig e Russell (2014)

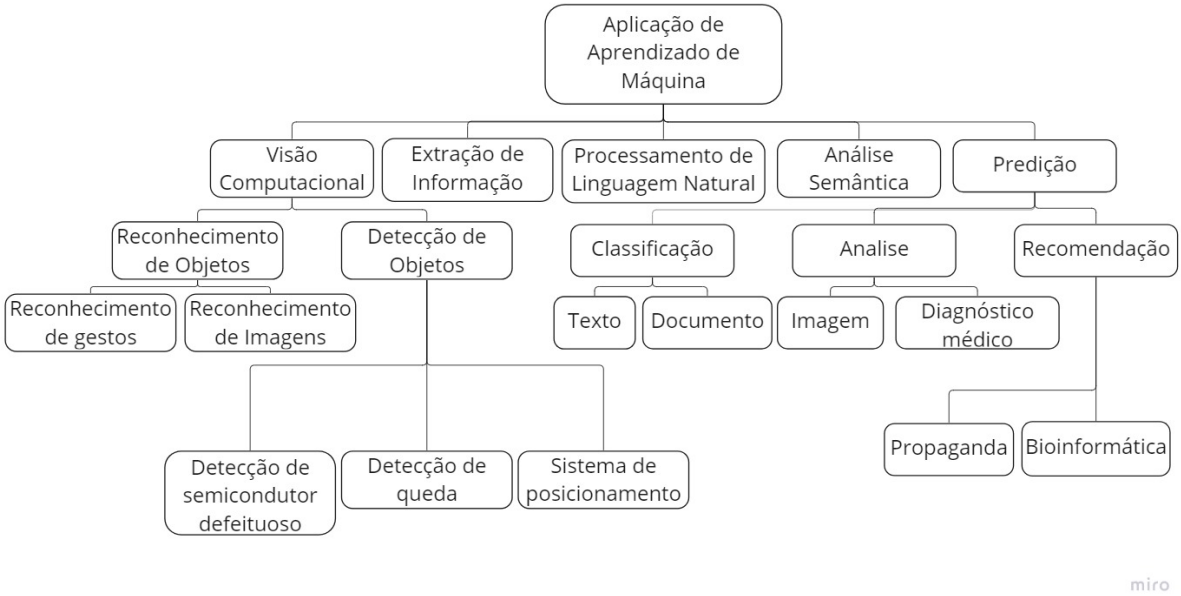
2.5.1 Aprendizado de Máquina

O aprendizado é a capacidade do indivíduo de se adaptar para melhorar seus resultados (NORVIG; RUSSELL, 2014). Porém, máquinas não são capazes de aprender como os seres humanos. O que é feito é achar uma fórmula matemática que, quando aplicado uma coleção de entradas, geram os resultados esperados. Essa abordagem não é

considerada um aprendizado como humanos fazem, pois caso as entradas sejam distorcidas, é muito provável que o resultado saia completamente errado. Mesmo com essas diferenças em definição, o termo aprendizado de máquina é reconhecido mundialmente, então o termo "aprender" é usado como analogia, ao invés da sua definição estrita (BURKOV, 2019).

Shinde e Shah (2018) lista algumas aplicações que o aprendizado de máquina possui e as coloca como um diagrama, como mostra a Figura 8: Visão Computacional, Predição, Análise Semântica, Processamento de Linguagem Natural e Extração de informações.

Figura 8 – Aplicações de Aprendizado de Máquina



Fonte: Adaptado de Shinde e Shah (2018)

Existem várias formas de fazer o treinamento de um modelo, as mais comuns são o aprendizado não supervisionado, o supervisionado e o aprendizado por reforço (HONDA; FACURE; YAOHAO, 2017) (TIBCO, 2022) (MAIER, 2020):

- O aprendizado supervisionado é o tipo de aprendizado em que os dados são fornecidos para o treinamento com a resposta do problema. A partir da entrada e da resposta o algoritmo busca padrões e semelhanças para resolver problemas de teste. As técnicas mais conhecidas para o aprendizado supervisionado são: *artificial neural networks*, *decision tree*, *k-nearest neighbors* e *naive bayes*;
- O aprendizado não supervisionado vai em direção oposta, nesse tipo de aprendizado o treinamentos são feitos com dados não rotulados e obrigam a modelo aprender por conta própria, com falta de informações básicas a máquina começa a buscar padrões e agrupar as entradas pela semelhança de cor, tamanho, forma etc. As técnicas mais conhecidas para o aprendizado não supervisionado são: *artificial neural networks*, *k-means*, *hierarchical clustering* e *gaussian mixture models* (GMM);

- Por fim o aprendizado por reforço que é o aprendizado através da experiência - meio similar com o processo de aprendizagem humana - que recompensa ou puni o agente pelas decisões tomadas. Espera-se que com a repetição do mesmo cenário o agente tome a melhor decisão, ou seja, aquela que forneça a melhor recompensa ou no pior dos casos a menor punição. Para mitigar o risco do modelo tomar sempre o mesmo caminho de escolas "ótimas" são aplicados incentivos para que o agente explore novas possibilidades durante o treinamento, buscando assim novas "ótimas" decisões. As técnicas mais conhecidas para o aprendizado por reforço são: *Q-learning*, *Temporal difference learning* e *Monte Carlo techniques*.

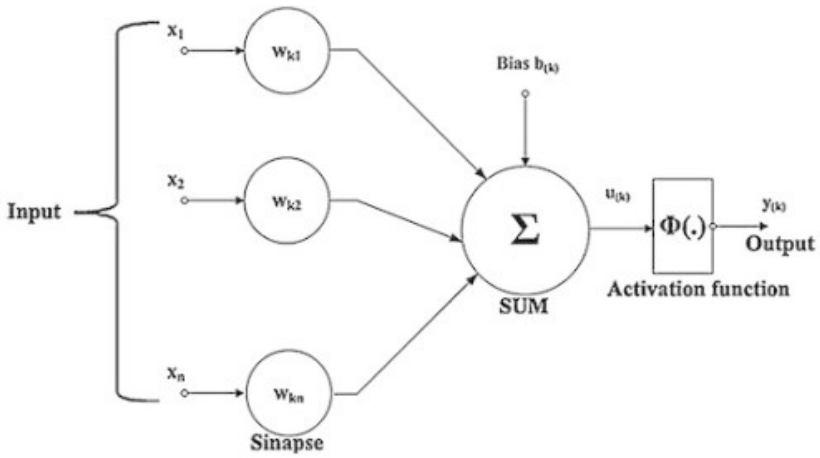
Na próxima seção serão abordadas as *artificial neural networks*, pois, além de serem base para alcançar o objetivo do trabalho, possibilitam uma maior gama de treinamentos, visto que podem ser usados tanto no treinamento supervisionado ou não supervisionado.

2.5.2 *Artificial Neural Networks*

As *Artificial Neural Networks* (ANN, redes neurais artificiais, tradução livre), ou simplesmente *Neural Networks* (NN), tem esse nome pois simulam o cérebro, tendo uma grande estrutura e capacidade de desenvolver suas próprias regras pela "experiência observada", ou seja, uma NN é projetada para modelar como o cérebro realiza uma tarefa particular (HAYKIN, 2001) .

De acordo com Academy (2022a) o cérebro artificial, assim como um biológico, é composto por neurônios, porém, um modelo matemático de neurônio, proposto por Warren McCulloch e Walter Pitts em 1943.

Figura 9 – Neurônio Artificial



Fonte: Academy (2022a)

Na Figura 9, os sinais de entrada x_n são provenientes de outros neurônios ou do ambiente. Essas entradas podem estimular mais ou menos o receptor de acordo com os

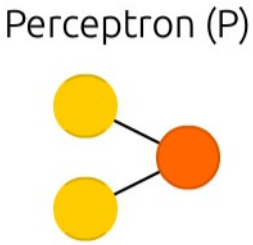
pesos sinápticos w_{kn}, quanto maior o peso, maior o estímulo. O corpo pode ser separado em duas partes: a primeira é o somatório dos estímulos (x_n) multiplicados pelo seu fator excitatório (w_{kn}); A segunda é uma função de ativação que definirá a saída do neurônio y_k entre excitatório quando o peso for positivo ou inibitório quando o peso for negativo. Essa função só é disparada caso o *threshold* (limiar) seja atingido (ACADEMY, 2022a).

Esse modelo considera que os valores, tanto de entrada, quanto de saída, são binários. McCulloch considerava que um neurônio influenciava ou não outro.

O próximo grande avanço nessa área foi o *perceptron*, fortemente influenciado pelo modelo de McCulloch e Pitts, e que é utilizado até os dias atuais (ACADEMY, 2022a) .

Frank Rosenblatt em 1957, introduziu o *perceptron*, a forma mais simples de uma rede neural artificial usada para a classificação de padrões, pois consiste de uma rede neural com apenas um neurônio artificial, sem nenhuma camada escondida (HAYKIN, 2001). Como mostra a Figura 10, os círculos amarelos são as entradas e o laranja é a saída.

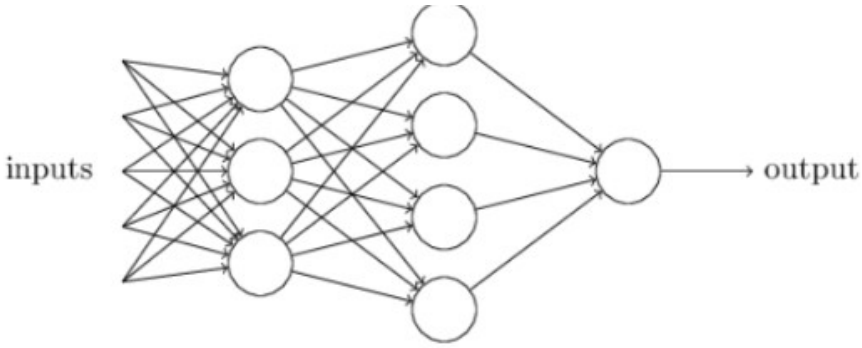
Figura 10 – *Perceptron*



Fonte: Asimov (2022)

Uma rede neural com mais de uma camada de perceptrons é conhecida como *Multi Layer Perceptron* - MLP (Perceptron de várias camadas, tradução livre), ou como ANN, e são compostas por uma camada de entrada, várias camadas escondidas e uma camada de saida, como mostra a Figura 11 (IBM Corporate, 2021).

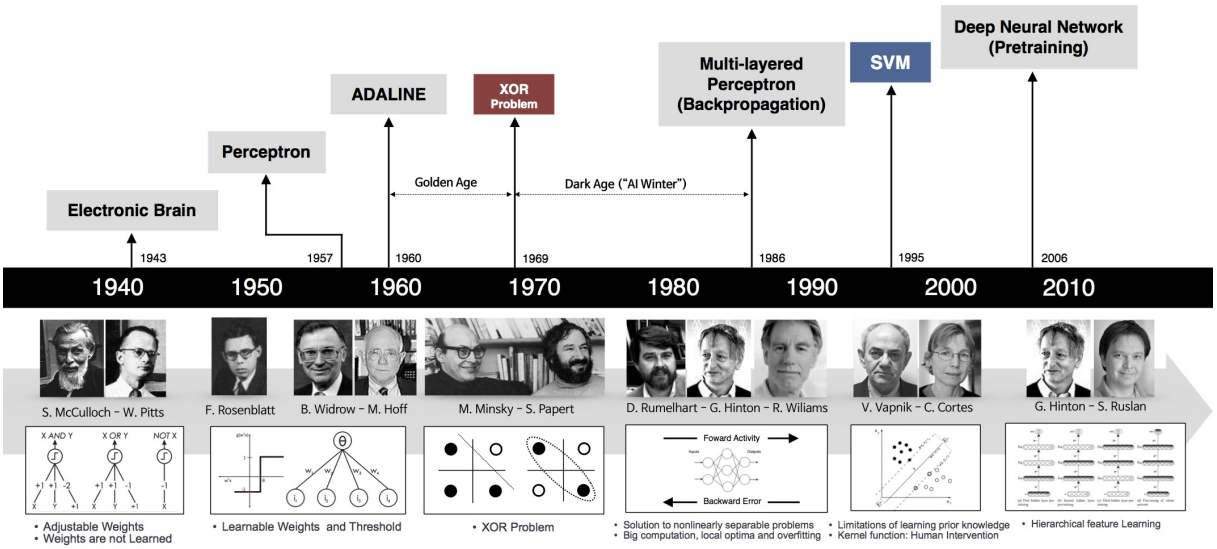
Figura 11 – Exemplo de ANN



Fonte:Academy (2022b)

Na Figura 12 é apresentado uma linha do tempo que mostra os marcos da ANN.

Figura 12 – Linha do tempo de marcos da ANN



Fonte: Academy (2022c)

2.5.3 *Deep Learning*

Segundo Kavlakoglu (2022) NN e *Deep learning* (aprendizado profundo, tradução livre) tendem a ser usados como termos intercambiáveis, o que pode causar confusão. O "profundo" do *Deep learning* se refere apenas à profundidade (quantidade) das camadas em uma NN. Nas palavras da IBM Corporate (2021): "Uma rede neural que consiste em mais de três camadas – que incluiria as entradas e a saída – pode ser considerada um algoritmo de aprendizado profundo. Uma rede neural que possui apenas duas ou três camadas é apenas uma rede neural básica."

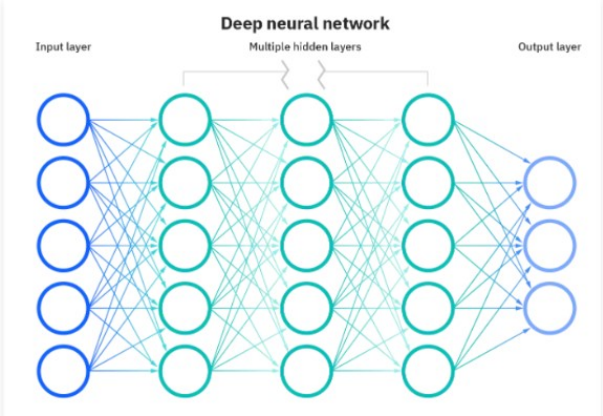
A Figura 13 mostra um exemplo de uma *Deep* NN.

Existem diversas classes de modelos de *deep learning* cada uma focada em sua especificidade, por exemplo, há os *Multilayer perceptrons*, *Autoencoders*, *Principal component analysis*, entre outras. A rede mais utilizada para captura de informações em uma imagem de entrada são as *Convolutional Neural Networks* (GOODFELLOW; BENGIO; COURVILLE, 2016).

2.5.4 *Convolutional Neural Networks*

Conhecidas como CNN - *Convolutional Neural Networks* (Redes Neurais Convolucionais), ou só *Convolutional Networks* (Redes Convolucionais), têm esse nome pois usam uma operação matemática conhecida como convolução no lugar de uma multiplica-

Figura 13 – *Deep NN*



Fonte: IBM Corporate (2021)

ção de matrizes comum em pelo menos uma das camadas (GOODFELLOW; BENGIO; COURVILLE, 2016).

Evitando o formalismo matemático a convolução é uma operação em duas funções, que produz uma terceira função que expressa como a forma de uma é modificada pela outra. Isso possibilita reduzir a entrada a um formato mais simples de processar sem perder recursos críticos (MANDAL, 2021).

CNNs combinam três ideias de arquitetura para garantir algum nível de mudança, escala e invariância de distorção: campos receptivos locais, pesos compartilhados ou reaplicação de pesos e subamostragem espacial ou temporal (LECUN et al., 1998) .

Uma CNN possui várias camadas como a camada de entrada, de convolução, *pooling*, ReLU, entre outras, e não é obrigatório utilizar todas elas. Dependendo do tamanho da entrada, a quantidade de camadas pode aumentar ou diminuir. Quanto mais profunda for a rede, melhor ela aprende, porém maior é o tempo de processamento (RAGHAVENDRA et al., 2018).

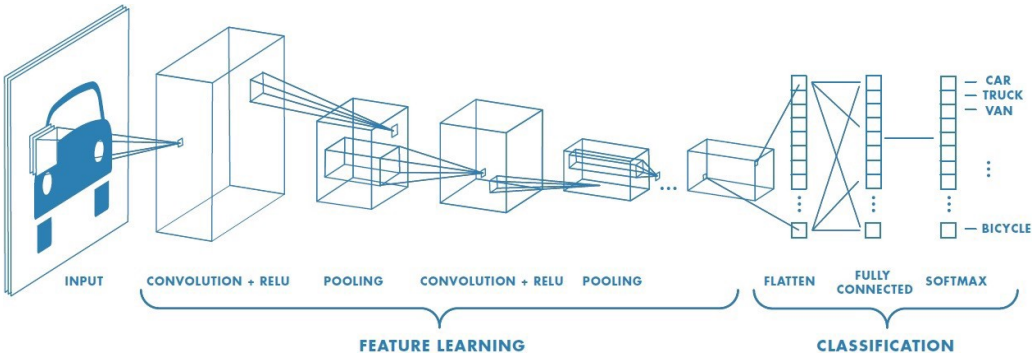
A Figura 14 mostra um exemplo de CNN e suas camadas

2.5.4.1 Camada de Convolução

É responsável por aplicar filtros convolucionais, chamados de *Kernel*, na entrada. Esse *kernel* varre a entrada toda, andando uma quantidade de valores definidos pelo usuário, chamado de *stride*, e extraem mapas de recursos (RAGHAVENDRA et al., 2018).

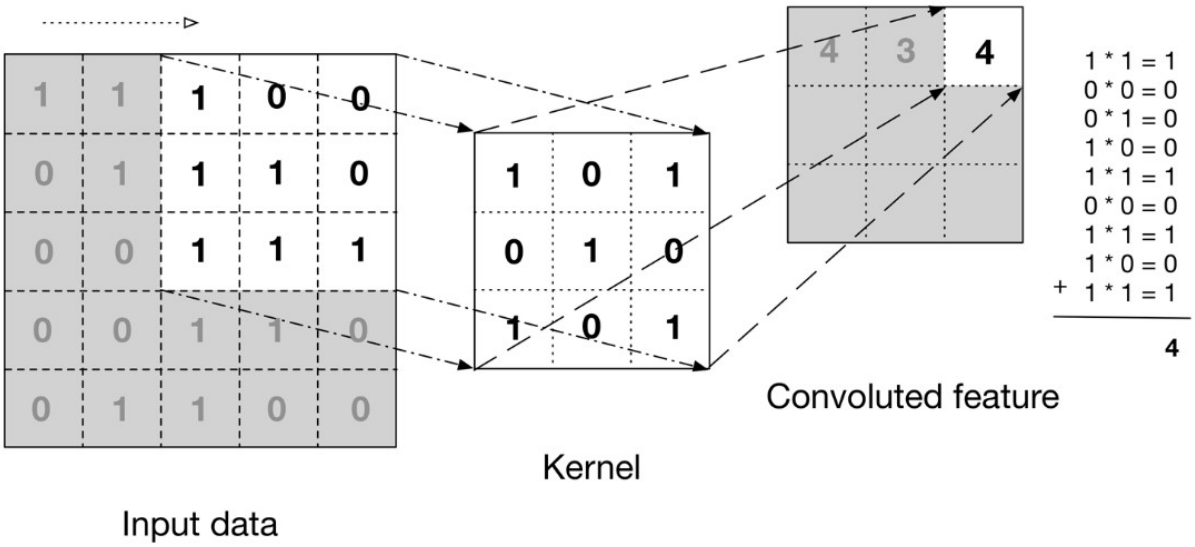
Na Figura 15 mostra um exemplo de operação na camada de convolução.

Figura 14 – *CNN e suas camadas*



Fonte: MathWorks (2022)

Figura 15 – *Exemplo de operação convolucional*



Fonte: Mandal (2021)

2.5.4.2 Camada de Normalização em Lote

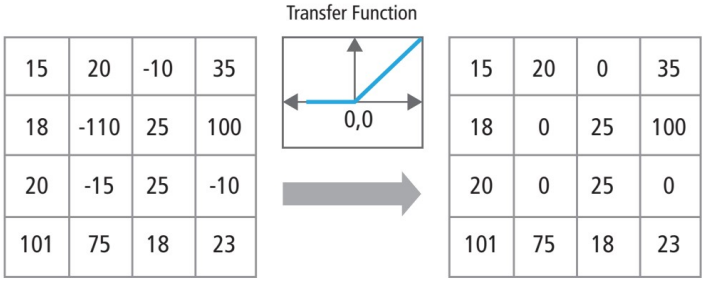
Essa camada realiza a normalização em cada mini-lote de treinamento, permitindo usar taxas de aprendizado muito mais altas e atingir a mesma precisão com 14 vezes menos etapas de treinamento (IOFFE; SZEGEDY, 2015) .

2.5.4.3 Camada de ReLU

Unidade Linear Retificada (ReLU) é uma função não linear e faz parte das camadas convolucionais e de normalização, na qual para cada elemento da entrada é realizada a operação de *thresholding* (limiarização), ou seja, valor menor que zero é definido como zero. Isso reduz a redundância de dados e mantém os recursos importantes intactos. O tamanho da saída desta camada é o mesmo da camada anterior (RAGHAVENDRA et al.,

2018). Um exemplo pode ser visto na Figura 16

Figura 16 – *Exemplo de operação ReLU*

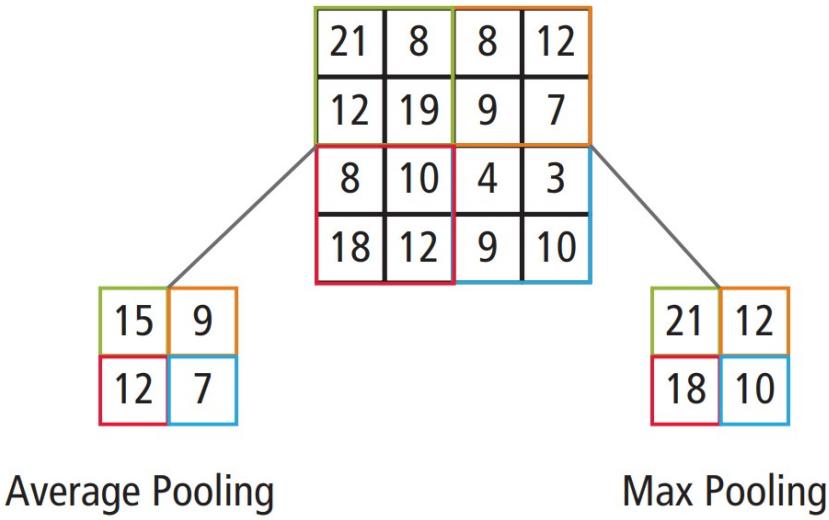


Fonte: Hijazi, Kumar e Rowen (2015)

2.5.4.4 Camada de *Pooling*

A camada de *Pooling* (subamostragem), reduz a resolução da entrada, tornando-os robustos contra ruído e distorção. Existem duas principais formas de criar novos mapas de recursos: a média (*average pooling*), na qual é feita a média dos valores; e o máximo (*max pooling*), no qual é selecionado o maior valor (HIJAZI; KUMAR; ROWEN, 2015). Um exemplo pode ser visto na Figura 17

Figura 17 – *Exemplo de operação Pooling*



Fonte: Hijazi, Kumar e Rowen (2015)

2.5.4.5 Camada totalmente conectada

As camadas totalmente conectadas são geralmente usadas como as camadas finais de uma CNN, são elas que realizam a soma ponderada da camada anterior. Todos os

elementos da camada anterior são usados no cálculo de cada elemento de saída (HIJAZI; KUMAR; ROWEN, 2015).

2.5.4.6 Camada de *Soft-max*

É uma função exponencial que normaliza os dados, ou seja, converte as amostras de dados para um intervalo de 0 a 1. Essa operação ajuda reduzir valores discrepantes, conhecidos como *outliers*, sem remover tais dados da amostra (RAGHAVENDRA et al., 2018).

2.6 Inteligência Artificial para acessibilidade

A maior parte das áreas derivadas da inteligência artificial tem um grande potencial para incluir pessoas com deficiências, trazendo liberdade e autonomia para esse grupo de pessoas e ajudando a tornar nossa sociedade mais igualitária e justa para todos (GUO et al., 2019). Nesta seção são abordados conceitos de reconhecimento de fala e visão computacional com o viés de auxiliar pessoas com deficiências visuais e motoras.

2.6.1 Reconhecimento de Fala

Os primeiros trabalhos sobre ASR datam da década de 1950, quando três pesquisadores da Bell Labs: Davis, Boddulph e Balashek, criaram em 1952 um sistema que reconhecia dígitos de 0 a 9 de um único usuário (LEVIS; SUVOROV, 2012).

Os primeiros sistemas de ASR implementavam a teoria da fonética acústica, que procurava padrões na fala e as comparava com os padrões ou modelos pré-armazenados no dispositivo. Essa tática funcionava bem ao nível de palavras isoladas, de itens foneticamente distintos em vocabulários pequenos. Porém, perdiam eficácia quando o vocabulário aumentava. Essa comparação entre a fala do usuário e os modelos pré-armazenados apresenta problemas se o comprimento de onda do falante e do armazenado forem diferentes, ou seja, esse modelo mostra dependência do usuário, o sistema só irá reconhecer palavras ditas por uma pessoa (RABINER; JUANG, 1993 apud LEVIS; SUVOROV, 2012).

Ainda segundo Levis e Suvorov (2012), Forgie e Forgie foram os primeiros que tentaram criar um sistema independente de usuário, em 1959. Posteriormente, cientistas experimentaram técnicas de normalização de tempo para minimizar diferenças entre a fala de diferentes locutores e detectar de forma mais confiável o início e fim da fala.

No meio da década de 70, o VIP-100 foi lançado, sendo o primeiro produto comercial que usava ASR. Era usado em processo simples como em empresas que fabricavam placas de televisões, para controle de qualidade. Porém, sua principal contribuição foi que ele influenciou a *Advanced Research Projects Agency* (ARPA), do departamento de defesa

dos Estados Unidos, a fundar a *Speech Understanding Research* (SUR), responsável por fazer e financiar pesquisas na área (JUANG; RABINER, 2004). Um dos produtos dessa pesquisa foi o Harpy.

O Harpy, desenvolvido pela universidade de Carnegie Mellon, conseguiu reconhecer um vocabulário de 1.011 palavras com uma precisão razoável (JUANG; RABINER, 2004). Usava uma série de otimizações e heurísticas que os criadores dele resumiram como:

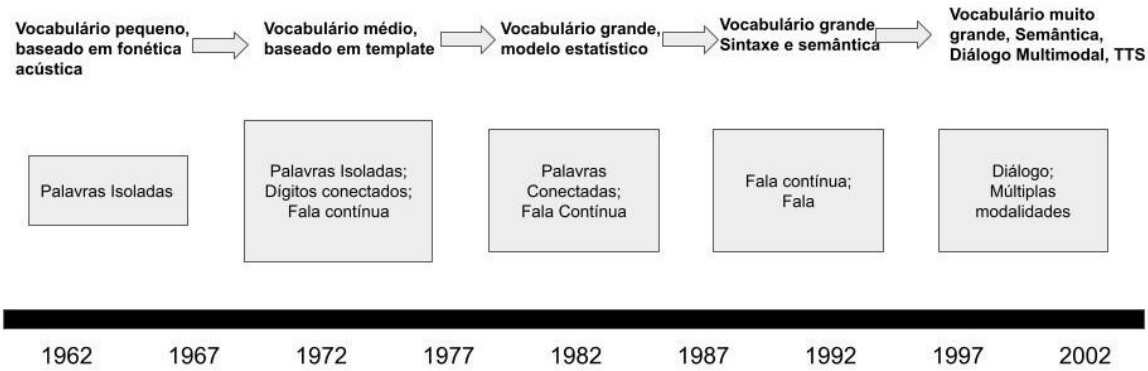
> "Harpy procura apenas alguns "melhores"caminhos sintáticos (e acústicos) em paralelo para determinar o caminho ótimo, e faz uso de segmentação para reduzir efetivamente o comprimento do enunciado, reduzindo a quantidade de atualizações de probabilidade que devem ser feitas."(tradução livre) (LOWERRE, 1976)

Um marco para os modelos de ASR veio apenas na década de 80, com o método estatístico *Hidden Markov Model* - HMM (Modelo oculto de Markov, traduzido do inglês) (MACEDO, 2017). O modelo tornou possível o aumentar vocabulário, pois ao invés de comparar o modelo de palavras com o padrão de som (padrão fonética acústica), ele representa as unidades da linguagem, por exemplo: fonemas ou palavras, como uma sequência de estados com probabilidades de transição entre cada estado. Para passar de um estado para outro, o modelo usará a transição mais alta de probabilidade (LEVIS; SUVOROV, 2012).

Nessa época também, os modelos de redes neurais também ganharam popularidade (MACEDO, 2017). Sua vantagem é sua classificação de padrões estáticos, o que é útil para detecção de palavras isoladas, mas não efetivos para reconhecimento contínuo (LEVIS; SUVOROV, 2012).

A Figura 18 mostra em uma linha do tempo os marcos na tecnologia de reconhecimento de fala.

Figura 18 – Marcos do ASR



Fonte: Adaptado de Juang e Rabiner (2004)

No anos 90, computadores com processadores mais potentes chegaram no mercado, viabilizando o ASR chegar na casa das pessoas. Porém, os produtos que os utilizam eram muito caros (PINOLA, 2011).

Em 2001, a ASR estagnou com 80% de precisão, e até o final da década não houve progresso aparente (MACEDO, 2017).

O conceito se popularizou quando as gigantes do mercado como a Google, Amazon, Apple, entre outras, lançaram seus *Voice-Activated Personal Assistants* (VAPAs, muito conhecido como assistentes virtuais). Estes produtos revolucionaram a forma com que as pessoas interagem com os seus celulares e aparelhos eletrônicos, inclusive pessoas com deficiência motora e visual que ganharam uma nova interface de interação (BRANHAM; ROY, 2019).

O mercado de ASR foi avaliado em 14.42 bilhões de dólares em 2021 (GRANDVIEWRESEARCH, 2022), aproximadamente 80 bilhões de reais, 1 dólar representando 5,5799 reais segundo BCB (2022); e tende a crescer muito nos próximos anos, existem estimativas para um crescimento de 15,3% até o ano de 2030 (GRANDVIEWRESEARCH, 2022).

Espera-se que o aproveitamento de algoritmos de *Deep learning* seja o fator-chave para o crescimento do segmento baseada em IA. O mercado é impulsionado pela crescente adoção de sistemas e dispositivos ativados por voz, e sistemas de assistentes virtuais em vários setores, como bancário, automotivo e de saúde, como também sua aplicação na área de acessibilidade (GRANDVIEWRESEARCH, 2022).

2.6.1.1 ASR para acessibilidade

Foi sugerido que uma das áreas mais promissoras para a aplicação do ASR é ajudar pessoas com deficiência (LEGGETT; WILLIAMS, 1984 apud NOYES; HAIGH; STARR, 1989).

Existem várias maneiras que o ASR pode ajudar, uma delas é permitir o controle de equipamentos, como é o caso do aplicativo Voice Access, no android, que permite que o usuário controle seu celular pela voz, disponibilizando comandos de navegação, como “Voltar”, “abrir aplicativo”; Controle de tela: “apertar botão”, “deslizar para baixo” e até Escrita de texto como "digite olá"e "substitua café por chá", possibilitando que pessoas com problemas motores utilizem o celular, por exemplo.

Wald (2005) propõe alguns exemplos de aplicação do ASR para deficientes auditivos:

Legendar vídeos ou aulas em tempo real para surdos ou deficientes auditivos. Fornecer notas de aula on-line automáticas sincronizadas com fala e slides, pois pessoas surdas e com deficiência auditiva e muitos outros alunos acham difícil ou impossível fazer anotações ao mesmo tempo que ouvem, assistem e pensam, enquanto outros não conseguem assistir à aula por problemas mentais ou motivos de saúde física.

2.6.2 Visão computacional

A *Computer Vision* (CV) do português visão computacional é uma das áreas da aprendizagem profunda que busca simular o funcionamento do olho e do cérebro humano responsável pela nossa habilidade de enxergar (GOODFELLOW; BENGIO; COURVILLE, 2016).

O estudo dessa área da computação iniciou após a metade do século XX, época conhecida como a revolução digital, um dos projetos iniciais que recebeu grande destaque foi o *The Summer Vision Project* desenvolvido por Papert (1966) que tinha a proposta de descrever o conteúdo de uma imagem determinando objetos e o plano de fundo da dela. Atualmente, a área de visão computacional teve uma grande evolução graças às redes neurais convolucionais que contribuíram para realizar processamento das imagens de forma mais eficaz e precisa em comparação aos métodos utilizados anteriormente (ULSON, 2018).

Através da aplicação da visão computacional, é possível fazer as seguintes tarefas com uma imagem (LI; JOHNSON; YEUNG, 2017) (TORRALBA et al., 2003):

- Classificar: Atribuir um rótulo para o elemento da imagem a partir de categorias pré definidas;
- Localizar: Encontrar a posição de um conteúdo na imagem;
- Detectar: Une os conceitos de classificar e localizar, podendo ser aplicado para diversos elementos de uma só vez;
- Detecção por contexto: Faz uso de familiaridade entre imagens para agregar antecedentes contextuais para reconhecer objetos;
- Segmentar por semântica: Categoriza cada pixel da imagem determinando se aquele pedaço faz parte de uma classe;
- Segmentar por instâncias: Categoriza elementos desejados na imagem determinando as regiões em que ele se encontra.

Os elementos identificados através de CV são objetos, ambientes, faces, textos, comportamentos, entre outros.

Como destacado por Dodge e Karam (2016), a deteção de objetos em uma imagem é uma tarefa não trivial devido a possíveis ruídos, baixa qualidade da imagem, luminosidade, ângulo, distorção e oclusão que podem atrapalhar até mesmo a identificação por um ser humano. Para minimizar esses problemas é comum a aplicação de conceitos de processamento de imagem para que a precisão do sistema seja maximizada.

2.6.2.1 Detecção de objetos

A detecção de objetos é uma ramificação do reconhecimento de objetos que através do escaneamento de uma imagem classificando os objetos e estimando suas posições (PATHAK; PANDEY; RAUTARAY, 2018).

Os principais desafios encontrados nesta área são em relação à qualidade da imagem que é resolvido através de processamento de imagem e a quantidade desconhecida de objetos a serem reconhecidos na imagem que é solucionado utilizando tipos de redes neurais específicas para detecção de objetos (WANGENHEIM, 2018), são elas *Region proposal based framework* e *Regression/classification based framework*.

2.6.2.1.1 *Region proposal based framework*

Region proposal based framework é um estilo de rede neural para detecção de objetos que funcionam em duas etapas, primeiro é feito uma análise geral da imagem em busca de regiões de interesse e posteriormente focaliza nestas regiões classificando-as (ZHAO et al., 2018).

Seguindo esse estilo, há muitos algoritmos, sendo um dos primeiros e o mais popular o *Region-based Convolutional Neural Networks* (R-CNN), que além da característica geral deste tipo de algoritmo, possui a necessidade de uma imagem de tamanho fixo como entrada (GIRSHICK et al., 2013), com objetivo de remover essa necessidade foi desenvolvido o *Spatial Pyramid Pooling Networks* (SPP) em 2014 (HE et al., 2014). A partir destes dois algoritmos foi criado o modelo *Fast Region-based Convolutional Neural Networks* (*Faster* R-CNN) com o objetivo de reduzir o tempo de treinamento e de teste, para fazer isso este algoritmo resolve as duas etapas de uma só vez utilizando uma camada de *Pooling* ao invés da camada de *Spatial Pyramid Pooling* (GIRSHICK, 2015).

Nos dias atuais tempos outros modelos que se baseiam no *Region proposal based framework* e se destacam, como por exemplo, o *Region-based Fully Convolutional Networks* (RFCN), *Feature Pyramid Networks* (FPN) e *Mask Region-based Convolutional Neural Networks* (*Mask* R-CNN) (ZHAO et al., 2018).

2.6.2.1.2 *Regression/classification based framework*

Regression/classification based framework é um modelo de rede neural que trata a detecção de objetos como um problema de regressão único e, diferentemente do *Region proposal based framework*, estas redes sempre consideram a imagem como um todo para realizar o processo. (ZHAO et al., 2018). Uma característica presente em algoritmos deste tipo - com algumas exceções - é a necessidade de imagens de tamanho fixo e a dificuldade para identificar objetos sobrepostos.

Um dos primeiros algoritmos seguindo essa ideia foi o DetectorNet um algoritmo que era capaz de identificar objetos (SZEGEDY; TOSHEV; ERHAN, 2013). Pouco tempo depois, Sermanet et al. (2013) criaram o OverFeat, um modelo que não precisa de uma imagem de entrada com o tamanho fixo e utiliza de janela multi-escala e janela deslizante para fazer a localização dos objetos. O modelo *Grid Convolutional Neural Network* (G-CNN) foi criado para fugir do convencional da época, ele é um algoritmo que não depende de criação de regiões candidatas e faz uso de *grids* para encontrar a localização dos objetos (NAJIBI; RASTEGARI; DAVIS, 2015). Aproveitando do princípio de *grid* Redmon et al. (2015) desenvolveram o *You Only Look Once* (YOLO) que analisa simultaneamente várias partes da imagem utilizando de um limiar de confiança para identificar as regiões da imagem que possui o objeto. Posteriormente, outras versões da YOLO foram desenvolvidas, sendo, a YOLOv2 (REDMON; FARHADI, 2016), YOLOv3 (REDMON; FARHADI, 2018), YOLOv4 (BOCHKOVSKIY; WANG; LIAO, 2020) e YOLOv5 (ZHU et al., 2021), em busca de aprimorar a velocidade e o poder de detecção. As diferenças entre versões dessa ferramenta serão abordadas mais adiante nesta seção.

Além desses modelos, existem outros que fazem uso de regressão/classificação para detectar objetos, sendo o MultiBox, AttentionNet, *Single-Shot Multibox Detector* (SSD) (LIU et al., 2016), *Deconvolutional Single Shot Detector* (DSSD) (FU et al., 2017) e *Deeply Supervised Object Detectors* (DSOD) (SHEN et al., 2017).

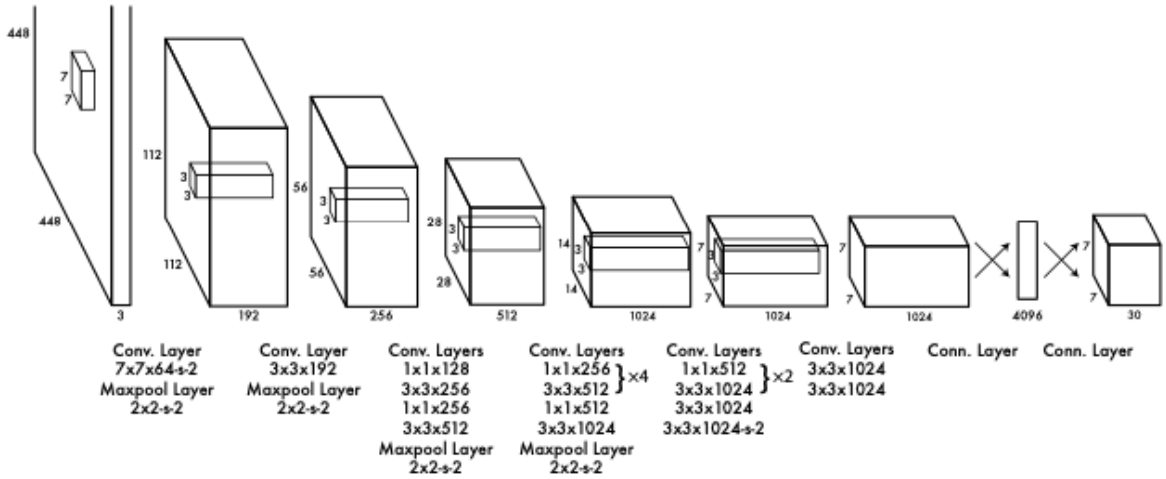
2.6.2.2 *You Only Look Once*

Como visto anteriormente, *You Only Look Once* (YOLO) é uma família de algoritmos utilizados para a detecção de objetos para imagens e vídeos e seu resultado é dado em tempo real. A sua primeira versão foi desenvolvida em 2015 por Redmon et al. (2015). Após essa versão, outros pesquisadores adicionaram outras funcionalidades e melhorias, criando assim, as demais versões. O motivo da sua velocidade de detecção se deve ao fato da imagem ser processada de uma vez, trazendo as ocorrências e localizações de forma eficiente. Para realizar isso, o algoritmo abre mão de precisão podendo ocasionar em perda de informação devido a dificuldade em encontrar objetos pequenos e erros na estimativa da posição (JIANG et al., 2022). A arquitetura original do algoritmo YOLO consiste de 24 camadas convolucionais seguidos de 2 camadas completamente conectadas como é possível ver na Figura 19, possibilitando que ele identifique apenas dois objetos que possuam a melhor estimativa dentro de uma unidade do *grid* (REDMON et al., 2015).

Segundo Jiang et al. (2022) as principais evoluções no algoritmo YOLO da v1 até a v5 são:

- YOLOv1: A divisão em *grid* responsável pela detecção, perda de confiança;
- YOLOv2: Âncora com *K-means*, treinamento em dois estágios, rede convolucional

Figura 19 – Rede neural da YOLOv1



completa, mudança do modelo de classificação para o Darknet-19 com 19 camadas convolucionais e 5 camadas de agrupamento máximo;

- YOLOv3: Detecção multi-escala usando *Feature Pyramid Network* (FPN) para resolver o problema com objetos pequenos, mudança do modelo de classificação para o Darknet-53;

- YOLOv4: *Spatial Pyramid Pooling* (SPP), função de ativação MISH, aprimoramento de dados *Mosaic/Mixup*, função de perda utilizando o sistema de estimativa *Generalized Intersection over Union* (IoU);

- YOLOv5: controle flexível do tamanho do modelo, aplicação da função de ativação Hardswish e aprimoramento de dados.

A YOLO é encarregada pela identificação de objetos na imagem de entrada, porém ela é comumente utilizada em conjunto com uma ferramenta capaz de fazer o processamento da imagem em busca de remover ruídos e melhorar o *dataset* para o treinamento e o teste do modelo (DODGE; KARAM, 2016). Existem diversas bibliotecas que fornecem recursos de processamento, como por exemplo, matlab, scikit-image, OpenCV, entre outras.

2.6.2.3 *Darknet*

Darknet é um *framework open-source* de alta performance para implementação de redes neurais, principalmente para detecção de objetos em tempo real. O motivo pela sua alta velocidade de treinamento e predição se deve por ter sido escrito em C e CUDA o que possibilita uma grande velocidade utilizando uma máquina com GPU (REDMON, 2013–2016). Existem algumas redes neurais que foram implementadas com a *Darknet* que servem como *backbone* para os modelos YOLO, a YOLOv2 implementa a Darknet-19, que

tem esse nome por causa das 19 camadas convolucionais (REDMON; FARHADI, 2016), a YOLOv3 faz uso da Darknet-53, que é mais potente, além das 53 camadas convolucionais usa blocos residuais para mitigar problemas com gradiente e ajudar a detectar objetos em diferentes escalas (REDMON; FARHADI, 2018) (SHARMA, 2022) e a YOLOv4 foi implementada com a CSPDarknet-53, que possui poucas diferenças em relação a sua predecessora, porém usa a estratégia de dividir e mergir os dados, para ter um fluxo maior de gradientes através da rede diminuindo a complexidade da amostra e mantendo a acurácia (BOCHKOVSKIY; WANG; LIAO, 2020) (TSANG, 2021).

2.6.2.4 OpenCV

OpenCV (*Open Source Computer Vision Library*, em português Biblioteca de visão computacional de código aberto) é uma biblioteca de código aberto que disponibiliza mais de 2500 algoritmos de visão computacional otimizados para ser utilizado em aplicações em tempo real (OPENCV, 2022). Inicialmente desenvolvido pela Intel em 1998, se transformou um projeto *open-source* em 2000 e possui mais de 18 milhões de downloads. É constituído por uma mistura de funções de processamento de imagens e algoritmos de alto nível de visão computacional, como por exemplo, processamento de imagem, detecção de face, detecção de pedestres entre outros (PULLI et al., 2012) (OPENCV, 2022). Construído com a linguagem de programação C e C++ e possui compatibilidade com os sistemas operacionais Linux, macOS, Windows e Android. É possível utilizar as funcionalidades do OpenCV utilizando outras linguagens, como por exemplo, Python, Ruby, Matlab, Java etc (OPENCV, 2022).

2.6.2.5 Visão computacional para acessibilidade

O uso da visão computacional para acessibilidade abre um grande leque de possibilidades, principalmente pensando no grupo de pessoas que possuem perda total da visão, como por exemplo, reconhecimento de faces, objetos, texto etc (GUO et al., 2019).

As aplicações da visão computacional se estendem por diversas áreas do dia-a-dia, por exemplo, segurança com sistemas de reconhecimento facial, saúde com a análise de exames mais assertivos utilizando a tecnologia, agricultura com a identificação e controle de pragas etc (LACERDA, 2021). Entretanto, a aplicação da visão computacional para a acessibilidade de pessoas com deficiências visuais pode ser muito explorada por empresas e pesquisadores, o fato desse público ter a visão comprometida abre grandes possibilidades para a criação de ferramentas assistivas que auxiliam a executar tarefas corriqueiras, por exemplo, se locomover em meio urbano (TONON, 2021), visitar museus (CROCE et al., 2020), consumir informações visuais na internet (FACEBOOK, 2022), fazer compras com segurança (FERREIRA, 2014) etc.

Grandes empresas já desenvolvem projetos e produtos com o foco em acessibilidade

usando visão computacional, como por exemplo, a *Envision Technologies*, que criou um aplicativo chamado *Envision AI* capaz de oferecer um infinidade de funcionalidades para pessoas com deficiência visual terem acesso a informações visuais de forma independente. Essa mesma empresa também desenvolveu um óculos inteligente chamado *Envision Glasses*, que analisa em tempo real o ambiente, objetos, pessoas e textos, descrevendo-os para o portador, trazendo uma visão de mundo para as pessoas que não enxergam (ENVISION, 2022).

2.7 Trabalhos relacionados

Esta seção busca apresentar alguns trabalhos relacionados com os assuntos descritos anteriormente: Visão computacional para acessibilidade e Reconhecimento de fala para acessibilidade.

Sant'ana (2014) desenvolveu um protótipo de controle por reconhecimento de voz por fala contínua que comandava um robô Lego NXT. A tecnologia utilizada para reconhecer os comandos de voz foi a LapsAPI, que é uma abstração da *engine* Julius, biblioteca de código aberto muito conhecida para reconhecimento de fala em tempo real. Neste projeto, 26 comandos foram mapeados através de uma gramática e implementados para a execução do robô, por exemplo, locomover ou locomover para direita, virar para a esquerda, mexa a cabeça etc. O funcionamento se dá pela seguinte sequência:

1. O usuário fala um comando para a interface (ex.: microfone plugado a um *notebook*);
2. A interface define qual comando o usuário deu de acordo com um grau de confiança;
3. A interface envia o comando para o robô através do *bluetooth*;
4. O robô interpreta o comando e executa a ação.

Para testar o projeto o autor chamou pessoas com características diferentes, como idade e sexo e o objetivo principal que era o reconhecimento dos comandos foi demonstrado, porém houve problemas com as esteiras responsáveis pela locomoção e o robô não conseguiu executar os comandos recebidos.

Já Macedo (2017) optou por desenvolver um módulo de reconhecimento de voz para a *game engine* Godot. Para isso, utilizou da biblioteca de reconhecimento de voz chamada *Pocketsphinx* desenvolvida em C, baseado em uma outra biblioteca chamada *CMUSphinx* que é feita em Java. O projeto possui uma abordagem mais generalista referente ao tratamento do comando falado em relação ao apresentado por Sant'ana (2014). O funcionamento do módulo é realizado de forma paralela ao jogo e o seu consumo é feito através de *buffer*, ou seja, quando um comando é identificado pelo módulo de

reconhecimento de fala, o texto é enfileirado em uma estrutura auxiliar, podendo ser consumido pelo jogo a qualquer momento. Por fim o autor implementou um simples jogo utilizando o módulo para demonstrar o funcionamento e teve um bom resultado quando exposto aos testes feitos com usuários reais.

De forma similar Lipka (2011) criou um *framework* generalista que faz uso de visão computacional e permite a interação com os jogos através de código simulando um ser humano observando a tela. Para isso, o autor utilizou Lua como linguagem de extensão para criar o *framework* que rodava com a *game engine* chamada *SpringRTS*, escrita em C++ e especializada em jogos de estratégia em tempo real. As funcionalidades criadas podem ser organizadas em *callOuts*, *input* e *vision*.

- *CallOuts* são chamadas internas feitas pela *game engine* para avisar que algum evento aconteceu, por exemplo, quando um *frame* novo é renderizado na tela o *callOuts.newFrame* é acionado ou então quando um botão deixa de ser pressionado com o *callOuts.buttonRelease*;
- *Inputs* são funcionalidades de entrada, por exemplo, é possível mover o cursor para as coordenadas x e y com o comando *input.moveCursor* ou um simples clique em uma região da tela com *input.clickOn*;
- *Vision* são as funcionalidades de visão computacional, com ela é possível encontrar ocorrência de imagens com o *vision.matchImage*, *templates* com *vision.matchTemplate* ou então obter todos os elementos que estão em primeiro plano com *vision.getForegroundObjects*.

Para os elementos de visão computacional, ele utilizou algoritmos prontos do OpenCV de acordo com o jogo, por exemplo, *Region proposal based framework* com SPP para detectar objetos, uma mistura de alguns algoritmos de subtração de plano de fundo e Canny para detecção de borda, contorno e planos. Como teste, o autor fez implementação de vários perfis de jogos, entre eles *Whack-a-mole*, *Breakout*, Tetris e Jogos de tiro em primeira pessoa, demonstrando que, com pequenos ajustes em relação aos métodos utilizados, o algoritmo era facilmente replicável para diversos estilos de jogos.

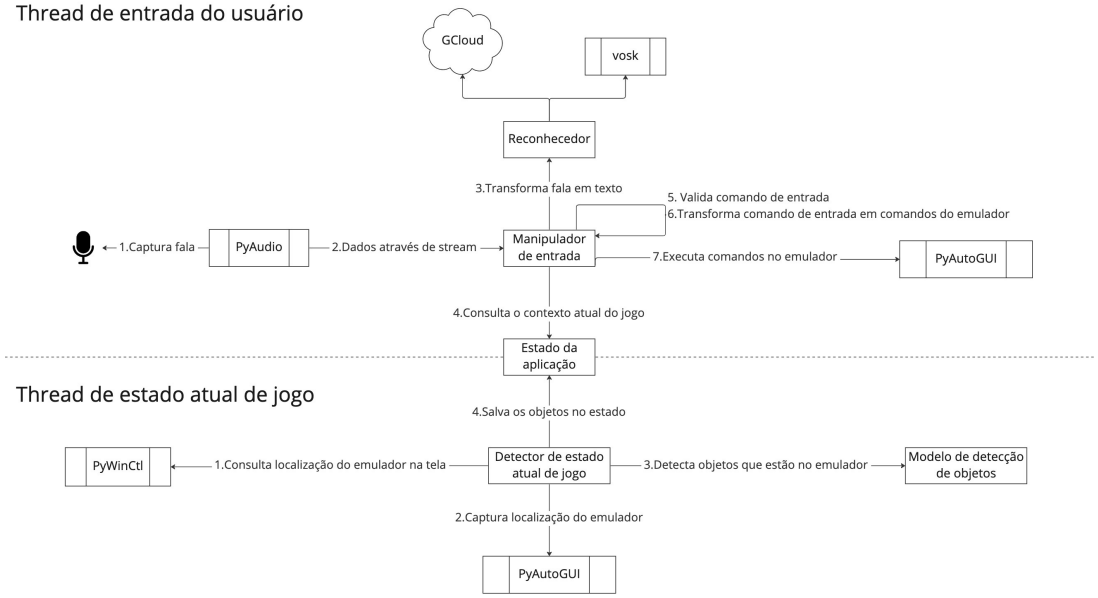
3 Desenvolvimento

Para atingir o objetivo principal do trabalho, que é promover maior acessibilidade em retro-jogos para pessoas com deficiência física ou motora, foi desenvolvido como prova de conceito um *software* que receba instruções por fala, interprete-as como ações do jogo, traduza elas para comandos do emulador e execute-os.

Será utilizada a linguagem *Python* para grande parte da solução, devido à abundância de bibliotecas e ferramentas utilizadas para criar e treinar modelos de detecção de objetos e reconhecimento de fala.

Este capítulo apresenta como foi o processo de implementação dessa solução, para atingir o objetivo do trabalho. A arquitetura final se encontra na Figura 20.

Figura 20 – Arquitetura



Fonte: Própria

3.1 Jogo escolhido

A franquia *Pokémon* foi escolhida por ser conhecida mundialmente e por ser do gênero RPG em turnos, uma característica importante para este trabalho, já que uma jogabilidade mais lenta possibilita testes sem a preocupação excessiva com o tempo de resposta do modelo.

O jogo *Pokémon Red* foi escolhido por ser o mais antigo da franquia, possui um conjunto de comandos e gráficos simples, o que contribui com o treinamento, além de não disponibilizar tantas opções de acessibilidade.

Outro motivo da escolha do *Pokémon Red* é que já foi usado em um evento conhecido como *Twitch Plays Pokémon* (TPP). Esse experimento ocorreu na *Twitch*, uma plataforma de *streaming*, em 2014, e consistia em transformar comandos enviados por usuários pelo *chat* do canal como "A", "B", "Esquerda"ou "Direita"em comandos no jogo. Usava ROMs e emuladores como base para os jogos e foi um evento bem-sucedido com 121 mil pessoas simultâneas e 55 milhões de visualizações no total (BULBAPéDIA, 2022).

3.2 Entrada do usuário

Para fazer a captura da fala do usuário, foi utilizada a biblioteca do Python chamada PyAudio, que faz um vínculo com o modulo *cross-platform* PortAudio v19 para entrada e saída de áudio. Através do uso da biblioteca, foram coletados os dados via *stream* e enviados para a ferramenta de reconhecimento de fala.

Para fazer o reconhecimento de fala foi criada uma classe abstrata com o objetivo de servir como interface de contrato para suportar diversas implementações. Durante o desenvolvimento, foram testadas as soluções apresentadas na seção 2.7, como também duas outras ferramentas, a biblioteca Vosk e o serviço do Google Cloud de *Speech to Text* (fala para texto).

Macedo (2017) utilizou a Pocketsphynx, uma biblioteca de código aberto e *offline*, mas que ela não possui modelo acústico para português, o que a descartou como opção viável.

Sant'ana (2014) utilizou a Engine Julius, uma Engine de código aberto e *offline*, mas como a Pocketsphynx, não possui modelo para o português, descartando também essa opção.

Durante a busca por outras opções que suportassem o português, a Vosk se mostrou promissora. Também, devido ao Pokémon não ser um jogo que precise de muitas ações por segundo, soluções comercias como da *Azure* e *Google Cloud* se mostraram opções viáveis, visto que a latência não é um fator crítico para o jogo, porém, cria a dependência de conexão com a *internet*.

Com a Vosk foram realizados testes utilizando modelos pré-treinados da comunidade que não garantiram um bom reconhecimento de palavras e frases simples, como, por exemplo, era comum a confusão da palavra "cima" com "acima", mesmo falando pausadamente. Ao pesquisar o que seria necessário para customizar e melhorar os modelos seria utilizado muito tempo, o que despriorizou essa solução.

Já com o serviço do Google Cloud de *speech to text* o reconhecimento funcionou melhor, além de possibilitar a customização do vocabulário, melhorando ainda mais o desempenho.

3.3 Capturador de tela

O capturador de tela foi implementado utilizando a biblioteca pyautogui, do Python, que disponibiliza interações com outros aplicativos, inclusive a captura a tela. Em conjunto, foi utilizada outra biblioteca, a PyWinCtl, que reconhece as posições das janelas abertas no computador. A imagem capturada é enviada para um *buffer* onde o detector de objetos faz o processamento e retorna os objetos encontrados no jogo naquele momento.

3.4 Detector de objetos

Para a criação do detector foi realizada em duas etapas: treinamento do modelo e utilização do mesmo pela solução.

3.4.1 Treinamento

Para treinar o modelo, é necessário um conjunto de dados para alimentar a rede para que ela aprenda. Não existia nenhum *dataset* pronto para o objetivo desse trabalho, logo, foi necessário montar um, com imagens do jogo, anotadas, ou seja, onde cada objeto está e o que ele é.

Para anotar as imagens, primeiramente, foi utilizado o aplicativo chamado *labelme*, uma ferramenta *Open Source* para anotação do dataset para treino. Com ela foi possível marcar as primeiras imagens utilizadas no modelo.

Ao pesquisar mais opções de ferramentas, foi encontrado o Roboflow, uma plataforma *online* para criação de *datasets* e treinamento de visão computacional. Possui uma ferramenta de marcação, parecida com o *labelme* e faz automaticamente a separação entre dados que são para treinamento e para teste, facilitando ainda mais o processo. Por disponibilizar esses recursos de graça, o Roboflow impõe a restrição que o *dataset* criado seja público na plataforma, podendo ser visto e utilizado por qualquer pessoa que acessar o *link*.

A plataforma disponibiliza a opção de treinar o modelo com o *dataset* criado e funciona a base de créditos, comprados com dinheiro. Cada treinamento custa 1 crédito, e quando se cria a conta são disponibilizados 3 créditos de graça, ou seja, é possível treinar 3 modelos sem gastar nada.

A primeira versão foi gerada a partir de capturas de tela do *Pokémon* sendo executado em um emulador. Foram 48 imagens e anotadas manualmente em agosto de 2022. O objetivo dessa primeira versão era que o modelo diferenciasse entre dois estados "principais" do jogo: em batalha e explorando. Após o treinamento, esse modelo ficou com 99% de precisão. A precisão alta dessa versão, mesmo com um número muito baixo de dados, se deve ao fato dos estados serem muito diferentes. A decisão de utilizar apenas 48 imagens se deve ao tempo estipulado para essa etapa, que era para ser rápida e servir de teste e de base para ajustar o arquivo de configuração do modelo.

Capturar a tela em alguns momentos não seriam suficientes para adicionar os objetos a serem reconhecidos na próxima versão do modelo. Foi necessário juntar mais imagens, mas como jogar o jogo todo para capturá-las não seria viável pelo tempo disponível, foi necessário buscar alternativas. Analisadas as opções disponíveis, foi decidido que seriam utilizados 2 vídeos na categoria *Longplay* que é apenas a tela do jogo sem a voz nem a câmera de quem joga, um do LongplaysHD (2019) e o outro do KryschnackLongplay (2021), ambos visitados em agosto de 2022. Após baixar o vídeo, foi usado o ffmpeg para retirar um *frame* do vídeo a cada 80 segundos.

```
1    ffmpeg -i "caminho para o video" -vf fps=1/80 out\%d.jpg
```

Foi decidido 80 segundos para retirar um *frame* do vídeo, pois, foi o cálculo para resultar 650 imagens utilizando o segundo vídeo como base. Mesmo que não seja um número alto, essa quantidade de imagens foi utilizado considerando o tempo para anotação manual e treinamento dessa nova versão. Caso o modelo não tivesse acurácia razoável, seria coletado mais *frames* em outros tempos para ter uma variedade de situações.

Utilizando o mesmo comando no primeiro vídeo, foi retirado apenas 150 imagens, pois, existia a preocupação com o tempo que levaria para realizar o pré-processamento delas, já que eram coloridas. Preocupação em vão, visto que ao enviar as imagens para o Roboflow, é possível adicionar opções de pré-processamento, como deixar as imagens em preto e branco, e a plataforma faz isso de forma muito rápida.

Depois de ter todos os *frames*, foram retiradas as imagens indesejadas, como telas totalmente brancas ou pretas, pois não possuem nenhum elemento reconhecível.

Com as imagens tratadas, foi utilizado o modelo treinado anteriormente para servir de base e acelerar o processo de treinamento. O Roboflow foi utilizado novamente para anotar as imagens novas e separar os dados de treinamento e adicionar ao dataset. A nova versão foi gerada em setembro de 2022, possui 797 imagens e reconhece 13 "objetos"com 85% de precisão.

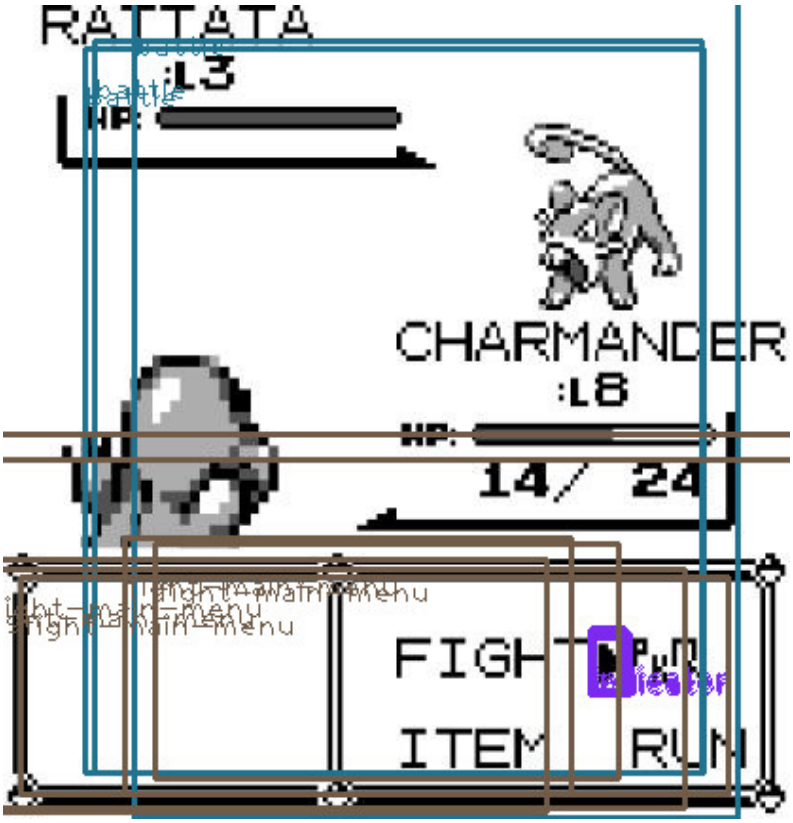
3.4.2 Aplicação

É responsável por realizar o reconhecimento dos objetos na tela e retorná-los. Utiliza o modelo treinado anteriormente, carregado e configurado pela função *readFromDarknet* do OpenCV.

Recebe a captura de tela e entrega ao modelo para que ele possa processar. Após o retorno do modelo, é feito um filtro em todos os resultados para remover os objetos com um nível de confiança baixo.

Com a lista de resultados filtrada, pode acontecer do modelo ter feito diversas predições para um mesmo objeto, como mostra Figura 21, então, foi necessário mais uma etapa que é a de mesclar os resultados de um mesmo objeto.

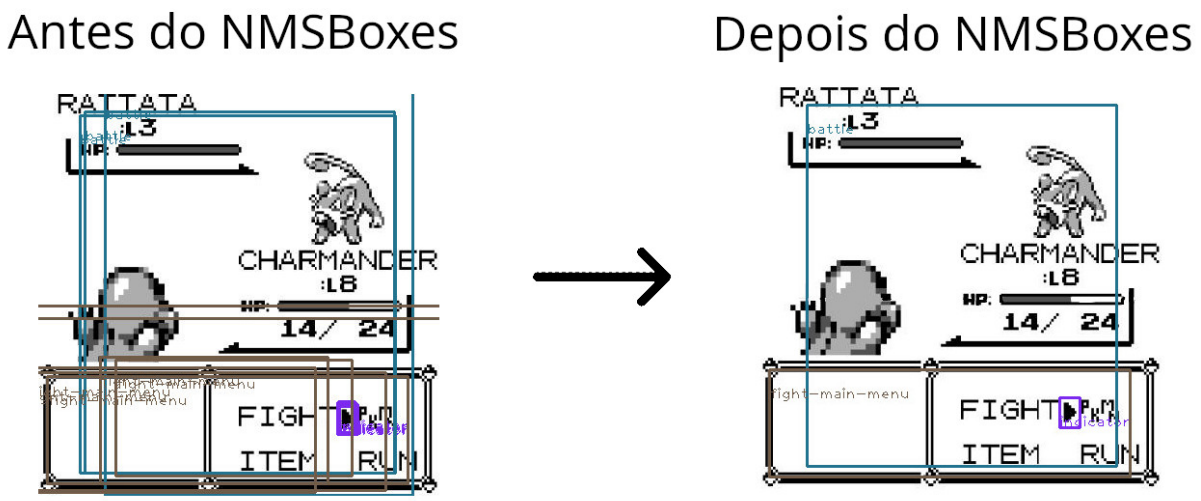
Figura 21 – Diversas predições no mesmo objeto



Fonte: Própria

A etapa de mesclagem é composta por uma função, chamada de *NMSBoxes*, que utiliza um algorítimo, chamado de *Non-max Supression*, que considera as classes e a intersecção entre elas. Após essa etapa, as predições foram ajustadas, como mostra a figura Figura 22. Feito isso, é retornado a lista de resultados, que possui o ID do objeto reconhecido, o nome, a posição, a altura e a largura.

Figura 22 – Predição antes e depois do NMSBoxes



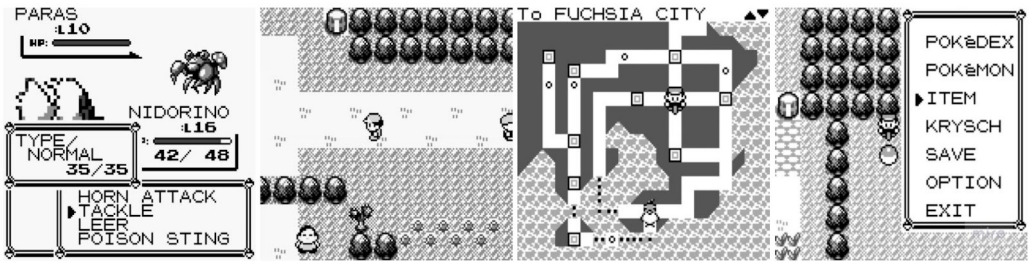
Fonte: Própria

3.5 Contextos

Durante o processamento da aplicação, um termo comumente utilizado é *contexts* (do português, contextos), que simboliza em qual momento do jogo o usuário está no momento. Os contextos demarcados foram: o de batalha, exploração, mapa e menu. Com essa definição, foi possível criar funcionalidades e comandos de voz para cada um desses. Para identificação, foram criadas as classes no modelo, onde suas anotações cobriam toda tela, deixando a mercê do modelo identificar os padrões e presumir qual o contexto que ele está recebendo como entrada.

Os contextos são utilizados para determinar quais são os comandos de entradas válidos e disponibilizar a implementação para os mesmos. Por exemplo, o contexto de batalha possui comandos diferentes do contexto de exploração. A Figura 23 mostra exemplos dos contextos escolhidos, em ordem: batalha, exploração, mapa e menu.

Figura 23 – Contextos escolhidos



Fonte: Própria

3.5.1 Contexto Genérico

O primeiro contexto que foi criado foi o genérico, pois, com ele já era possível fazer os comandos básicos do Game Boy (esquerda, cima, direita, baixo etc) e interagir com o jogo com pouco esforço. O objetivo dele era criar o contrato para os demais contexto e reaproveitar a lógica dos comandos do videogame através da herança dessa classe.

Numa classe de contexto há 2 métodos públicos: *get_valid_tokens*, responsável por dizer quais comandos de entrada são válidos para o atual contexto de jogo e *get_commands*, que recebe um *token* válido e retorna os comandos do emulador que são necessários para executar o comando de entrada.

3.5.2 Contexto de Batalha

Durante o desenvolvimento do trabalho, foi decidido dar mais foco no contexto de batalha por ser a parte fundamental do jogo e a que mais se repete durante a jogatina. Para utilizarmos o conceito de contexto foram criadas três novas classes, compondo os subcontextos de batalha. Eles são o menu principal de batalha, o menu de ataques e o menu da mochila. Os subcontextos são necessários visto que cada menu possui comportamentos e implementações específicas. Por exemplo, o menu de ataque tem suas opções empilhadas e é cíclico, se o usuário está no último ataque e entra com mais uma instrução para baixo, vai para o primeiro ataque, diferentemente do menu principal, que dispõe suas opções em formato de *grid* e não é cíclico.

Usando como exemplo o menu principal, há implementação para os comandos mais baixo nível do Gameboy, sendo esquerda, cima, direita, baixo, *ok*, seleciona e cancela. Também existem comandos de mais alto nível, como as opções de lutar, correr, mochila e lista de pokémons.

Para mover para a opção correta, foi necessário a criação de uma nova classe para representar o indicador do menu na tela. Para tanto, se criou um mapeamento das possíveis posições onde o indicador se encontra na tela e calcular quais comandos eram necessários para chegar na opção desejada.

A Figura 24 demonstra as anotações feitas para o treinamento do modelo. A borda verde representa a classe de batalha (*battle*), o azul claro delimita a classe do menu principal de batalha (*fight-main-menu*) e o rosa é o indicador (*indicator*).

3.6 Validação de comandos do usuário

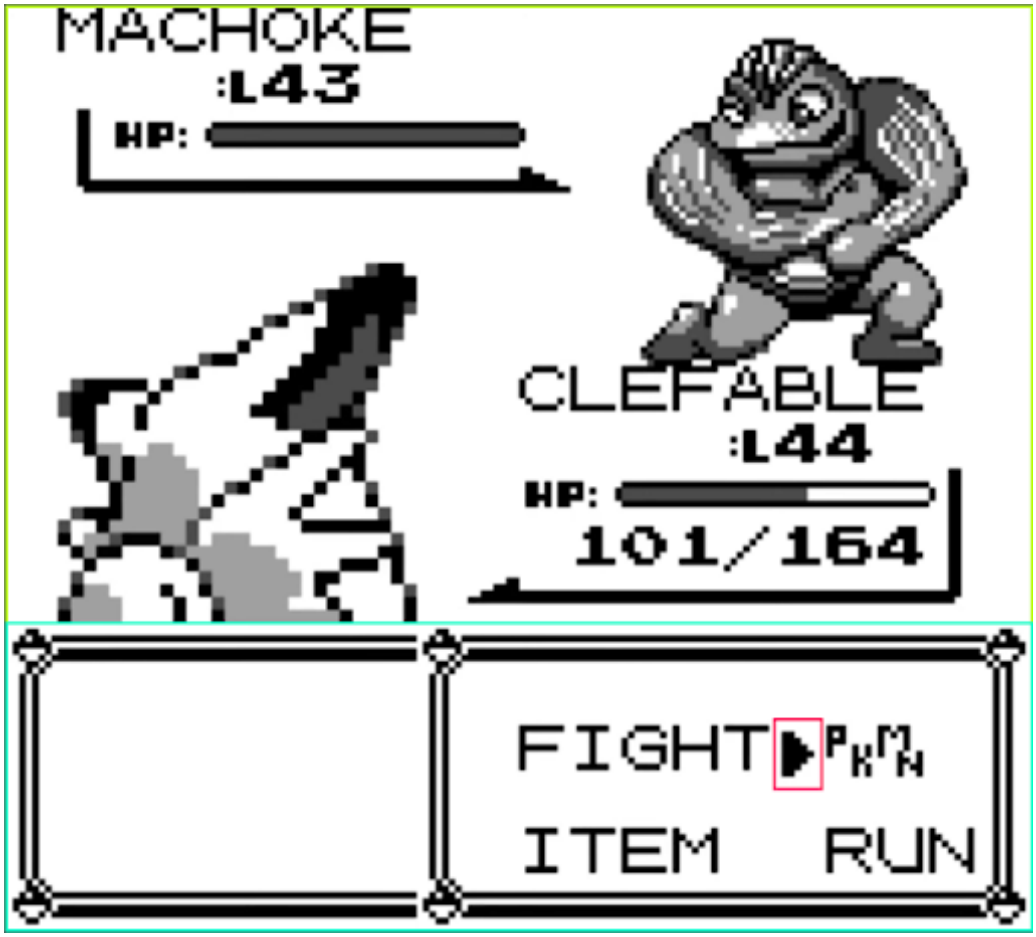
Quando o modelo identifica o contexto atual, qualquer palavra dita pelo usuário é validada através de uma verificação é simples. Verifica-se que o resultado do reconhecimento está entre os comandos válidos para aquele contexto e, caso não esteja, nenhuma ação

será executada e o usuário será avisado do comando inválido. Por exemplo, se o jogo está no mapa e o comando "atacar"é falado, o comando será ignorado e o usuário receberá a notificação de entrada inválida.

3.7 Comandos do emulador

Como resultado da função *get_commands*, implementado nos contextos, um comando de usuário é dado como entrada e o retorno são comandos de emulador necessários para realizar aquela ação. Por fim, esses comandos são executados utilizando as funções de controle de teclado do módulo PyAutoGUI. Com a conclusão do processo, o usuário pode fornecer outro comando como entrada e o fluxo se repete, gerando mudanças no estado atual do jogo.

Figura 24 – Classes anotadas na ferramenta do Roboflow



Fonte: Própria

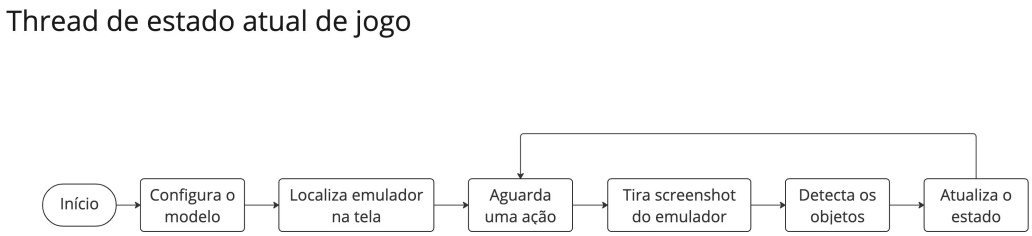
3.8 Integrando as partes do sistema

No momento de integração das partes, foi idealizado uma estrutura paralela utilizando *multithreading* do Python, para que não exista nenhum tipo de atraso ou bloqueio de execução em um dos processos. Na solução, são executadas duas *threads* sendo elas a de estado atual de jogo e a de entrada de usuário.

3.8.1 *Thread* de estado atual de jogo

Essa *thread* é iniciada com o processo principal da aplicação, com o objetivo de coletar as informações que estão em tela através de sua captura, utilizando a imagem como entrada para predição do modelo. Por fim, o resultado desse processo é estruturado em um estado em memória, constituída por quais objetos estão disponíveis para interação. O fluxograma que demonstra esse processo se encontra na Figura 25.

Figura 25 – Fluxograma: *Thread* de estado atual de jogo



Fonte: Própria

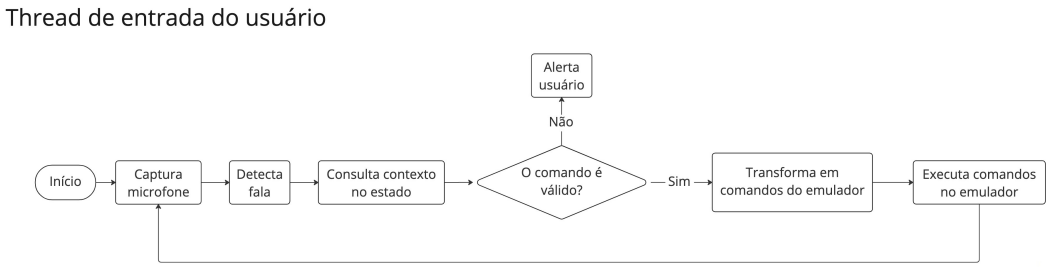
3.8.2 *Thread* de entrada do usuário

A *thread* de entrada do usuário também é iniciada com o processo principal, porém, abre um *streaming* que capta o microfone do usuário. Essa informação é usada pelo serviço ou biblioteca de ASR para retornar a palavra ou frase reconhecida, verificar se o comando de entrada é válido no estado atual de jogo e converter o comando de entrada em comandos do emulador. O fluxograma deste processamento está disponível na Figura 26.

3.8.3 Comunicação entre *threads*

Com essa solução, houve a necessidade de criar um meio de comunicação entre as partes, visto que a *thread* de entrada do usuário precisa saber quais objetos estão na tela para validar se o comando é possível. Para isso, foi utilizada uma solução de estado compartilhado em memória, através de um *Singleton*, onde a *thread* de estado atual de

Figura 26 – Fluxograma: *Thread* de entrada do usuário



Fonte: Própria

jogo possui a permissão de escrita e a *thread* de entrada de usuário consegue apenas fazer a leitura desse estado.

Fora a comunicação, também foi necessário encontrar um bom momento para atualizar o estado atual de jogo, que poderia ser um intervalo de tempo ou um comando dado pelo usuário. Após discussões, a primeira abordagem foi descartada, já que a escolha de qualquer valor seria arbitrária e possivelmente não conseguiria atender ao requisito principal: manter o estado sempre atualizado. Portanto, a abordagem de ação do usuário foi escolhida, o que foi feito através da implementação de um sistema de semáforo capaz de fazer com que *thread* de estado atual do jogo espere até que um comando de entrada seja executado no emulador, de forma que o estado do jogo mude e um novo ciclo seja iniciado.

4 Testes e resultados

Para validar a usabilidade e aplicabilidade do *software* produzido, foi organizado em sala de aula uma sessão de teste, na qual participaram sete pessoas, totalizando aproximadamente setenta e quatro minutos de teste de usabilidade.

No processo foi utilizado um computador do Centro Universitário Senac Santo Amaro e com um *headset*, durante um encontro de TCC, ou seja, existia ruídos oriundos de conversas paralelas e um chiado considerável emitido pelo microfone.

Foi gravado uma média de dez minutos de jogatina por usuário, na qual os jogadores ficavam livres para realizar as ações que desejassem. Os comandos foram separados em duas categorias: em exploração e em batalha, com o objetivo de compilar e analisar as seguintes informações:

- Assertividade no reconhecimento de fala.
- Assertividade na detecção do contexto do jogo.

Vale ressaltar que esse grupo de pessoas abrangiam os sexos masculino e feminino, além de contar com pessoas que tinham ou não experiência com o jogo. Infelizmente, não foi possível realizar testes com pessoas que possuem deficiências físicas ou motoras, o público alvo da solução.

4.1 Resultados

Nesta seção serão apresentados os resultados colhidos para assertividade no reconhecimento de fala e assertividade na detecção do contexto do jogo.

4.1.1 Assertividade no reconhecimento de fala

Os dados estão organizados anonimamente, pois não foi solicitado a autorização dos participantes, o resultado da assertividade no reconhecimento de fala é demonstrado na Tabela 1.

Durante todo o teste, foram realizados 321 comandos em exploração, sendo que 205 foram reconhecidos de forma correta e 116 de forma incorreta, totalizando 63,86% de assertividade. No contexto de batalha, foram 227 comandos, sendo que 184 foram reconhecidos corretamente e 43 erroneamente, totalizando 81,06% de assertividade.

Tabela 1 – Assertividade no reconhecimento de fala da solução

	Exploração			Batalha		
	Certo	Errado	Assertividade	Certo	Errado	Assertividade
Usuário 1	104	58	64,20%	35	12	74,47%
Usuário 2	13	0	100,00%	35	6	85,37%
Usuário 3	55	41	57,29%	12	1	92,31%
Usuário 4	14	2	87,50%	29	14	67,44%
Usuário 5	14	13	51,85%	26	5	83,87%
Usuário 6	2	1	66,67%	20	2	90,91%
Usuário 7	3	1	75,00%	27	3	90,00%

4.1.2 Assertividade na detecção do contexto do jogo

Como resultado dessa coleta houve uma boa assertividade na detecção, como não existia a implementação do contexto de exploração 100% das vezes em que o jogador se encontrava explorando o contexto identificado era o genérico, que possui os comandos básicos do emulador.

No que diz respeito ao contexto de batalha, a assertividade se aproximou dos 100%, com apenas uma falha no momento na tela de seleção de pokémons, onde o usuário 1 deu um comando válido e o mesmo foi executado de forma errada, pois, foi identificado o contexto errado.

4.2 Análise de resultados

Foi possível ver a diferença entre a quantidade de comandos efetuados nos dois contextos. Isso se deve ao fato da batalha estar mapeada, como, por exemplo, o comando de fugir, que pode economizar de dois a três comandos por fala. O que não acontece na exploração, onde um mapeamento de “manter a direita” ou “andar em círculos"auxiliaria a reduzir a quantidade de comandos necessários para executá-la.

A quantidade de erros nos comandos de exploração teve como principal causa os gêneros das palavras escolhidas. "Direita", por exemplo, foi erroneamente reconhecida como "direito". Caso fosse implementado um algoritmo para reconhecer uma palavra com X diferenças de ortografia o problema poderia ter sido mitigado.

Outro fator prejudicial analisado nas gravações foi o volume do ruído e da fala do usuário, que afetava o reconhecimento do que foi falado. Alguns exemplos que ocorreram foram: o reconhecedor não saber quando um comando terminou, pois, o ruído continuava e nada era computado, pois, o receptor considerava o ruído como parte da fala. O ruído também afetava quando o usuário falava em um tom mais baixo, ocasionando o reconhecimento de apenas uma sílaba da palavra.

O fato da solução reconhecer palavras isoladas, ao invés de frases com os comandos contextualizados, dificulta o trabalho do reconhecedor ao lidar com palavras pequenas e que possuem fonéticas parecidas, como, por exemplo, a confusão de "clima" com "cima". Outra possibilidade de solucionar esse problema é a criação de um vocabulário próprio para a solução.

As sessões de testes mostraram que, em momentos de exploração, a experiência era mais frustrante, justamente pela quantidade de comandos necessários, já que não existia implementação de comandos específicos deste contexto, além dos erros de palavras isoladas, seja pelo gênero reconhecido, seja pela fonética parecida.

5 Conclusão

Este trabalho teve como objetivo implementar uma forma de controle por fala em retro-jogos 2D, do gênero RPG por turnos e executada a partir de um emulador para promover uma maior acessibilidade, especialmente de pessoas com deficiência física ou motora. Para isso foi desenvolvido um software como prova de conceito, responsável por capturar os comandos falados pelo usuário e fazer seu processamento, até ser executado no emulador.

A execução desse projeto pode ser considerado um sucesso, pois foram realizados testes com a solução e foi possível comprovar a viabilidade de jogar jogos do estilo RPG por turnos através da fala. Pode-se considerar também que, com a arquitetura criada para o jogo *Pokémon Red*, é possível criar implementações para demais jogos do mesmo estilo.

O treinamento do modelo foi a parte que mais demorou, já que não foi encontrado nenhum *dataset* pronto. Durante a geração do *dataset*, diversas abordagens foram discutidas, mas foi encontrada uma maneira inteligente e rápida de coletar as imagens com os de vídeos, como foi explicado na subseção 3.4.1. Entretanto, as anotações das classes foram bem trabalhosas e geraram bastantes dúvidas a respeito do que era importante ser identificado.

A idealização do conceito de contextos foi um ponto crucial para a implementação, pois através dele houve a possibilidade de criar comandos do usuário para cada parte do jogo e implementações específicas para eles. Para a replicação desta arquitetura para outros jogos do mesmo gênero, se faz necessário o processo de geração de *dataset*, a idealização e anotação das classes e a criação dos contextos.

A utilização do PyAutoGUI para a entrada de comandos possibilita a utilização de emuladores diferentes, aumentando a gama de consoles que a solução abrange, e, consequentemente, mais jogos.

No momento que o *software* foi colocado à prova, constatou-se a necessidade de melhorias para garantir uma melhor experiência de usuário. As melhorias foram identificadas nos módulos de reconhecimento de fala e de gerenciamento de contextos.

Esse trabalho contribui para a Ciência da Computação trazendo a tona um assunto importante e atual: como a área e seus conceitos conseguem diminuir a desigualdade enfrentada pelas pessoas que possuem deficiências e traz consigo uma solução que pode ser aplicada não só para jogos, mas para diversas atividades do dia-a-dia.

5.1 Trabalhos futuros

Como visto na seção 4.2 e no Capítulo 5, o software possui diversos pontos de melhoria. Alguns exemplos são:

- Focar no reconhecimento de fala para mitigar os erros de palavras isoladas, criando um modelo específico para uma gramática pré definida.
- Possibilitar entrada em outras línguas, por exemplo, inglês, com o objetivo de incluir mais pessoas.
- Criar implementações para os demais contextos, principalmente para o de exploração e mapa. Com base no que foi observado nos testes e resultados.
- Aplicar o uso de texto para fala, para incluir pessoas com deficiência visual.
- Evoluir o uso de ataques, possibilitando ao usuário falar o nome do movimento ao invés do número da opção.
- Evoluir a mudança de pokémon, possibilitando que o usuário possa falar o nome do pokémon que deve entrar em batalha.
- Testar exaustivamente a solução para o jogo, utilizando apenas a fala para certificar que a implementação possibilita o usuário finalizar o jogo.

Referências

ABLEGAMERS. *History of Adaptive Tech*. 2021. <https://ablegamers.org/history-of-accessibility-in-gaming/>. [Online; accessed 29-May-2022]. Acesso em: 29 may. 2022. Citado 2 vezes nas páginas 19 e 28.

ACADEMY, D. S. *Deep Learning Book*. 2022. Disponível em: <https://www.deeplearningbook.com.br/o-neuronio-biologico-e-matematico/>. Acesso em: 21 mai. 2022. Citado 2 vezes nas páginas 32 e 33.

ACADEMY, D. S. *Deep Learning Book*. 2022. Disponível em: <https://www.deeplearningbook.com.br/o-perceptron-parte-1/#:~:text=Perceptron%20%C3%A9%20uma%20rede%20neural,os%20dados%20de%20entrada%20fornecidos.> Acesso em: 21 mai. 2022. Citado na página 33.

ACADEMY, D. S. *Deep Learning Book*. 2022. Disponível em: <https://www.deeplearningbook.com.br/uma-breve-historia-das-redes-neurais-artificiais/>. Acesso em: 21 mai. 2022. Citado na página 34.

ASIMOV. *The Neural Network Zoo*. 2022. Disponível em: <https://www.asimovinstitute.org/neural-network-zoo/>. Acesso em: 21 mai. 2022. Citado na página 33.

BARKER, S. *Horizon: Zero Dawn Cost a Reported €45 Million to Make*. 2017. Disponível em: <https://www.pushsquare.com/news/2017/03/horizon_zero_dawn_cost_a_reported_e45_million_to_make>. Acesso em: 31 mar. 2022. Citado na página 22.

BARRIO, V. M. D.; FERNÁNDEZ, A.; UPC, F. Study of the techniques for emulation programming. In: . [s.n.], 2001. Disponível em: <http://www.xsim.com/papers/Bario.2001.emubook.pdf>. Citado 3 vezes nas páginas 24, 25 e 26.

BCB, B. C. do B. *Conversor de Moedas*. 2022. Disponível em: <https://www.bcb.gov.br/conversao>. Acesso em: 27 mai. 2022. Citado 2 vezes nas páginas 22 e 40.

BOCHKOVSKIY, A.; WANG, C.-Y.; LIAO, H.-Y. M. *YOLOv4: Optimal Speed and Accuracy of Object Detection*. arXiv, 2020. Disponível em: <https://arxiv.org/abs/2004.10934>. Citado 2 vezes nas páginas 43 e 45.

BOLTJES, E. *Horizon Zero Dawn: A Game Design Postmortem*. Estados Unidos: Game Developers Conference, 2018. Disponível em: <https://youtu.be/TawhcWao9ls>. Citado na página 22.

BRANHAM, S. M.; ROY, A. R. M. Reading between the guidelines: How commercial voice assistant guidelines hinder accessibility for blind users. In: *The 21st International ACM SIGACCESS Conference on Computers and Accessibility*. New York, NY, USA: Association for Computing Machinery, 2019. (ASSETS '19), p. 446–458. ISBN 9781450366762. Disponível em: <https://doi.org/10.1145/3308561.3353797>. Citado na página 40.

BULBAPéDIA. *Twitch Plays Pokémon*. 2022. Disponível em: <https://bulbapedia.bulbagarden.net/wiki/Twitch_Plays_Pok%C3%A9mon>. Acesso em: 27 out. 2022. Citado na página 50.

BURKOV, A. *The Hundred-Page Machine Learning Book*. Andriy Burkov, 2019. ISBN 9781999579517. Disponível em: <http://ema.cri-info.cm/wp-content/uploads/2019/07/2019BurkovTheHundred-pageMachineLearning.pdf>. Citado na página 31.

CHEIRAN JEAN FELIPE PATIKOWSKI E PIMENTA, M. S. "eu também quero jogar!": Reavaliando as práticas e diretrizes de acessibilidade em jogos. In: *Proceedings of the 10th Brazilian Symposium on Human Factors in Computing Systems and the 5th Latin American Conference on Human-Computer Interaction*. Porto Alegre, BRA: Brazilian Computer Society, 2011. (IHC+CLIHC '11), p. 289–297. ISBN 9788576692577. Citado na página 27.

CROCE, D. et al. Supporting autonomous navigation of visually impaired people for experiencing cultural heritage. In: _____. [S.l.: s.n.], 2020. p. 25–46. ISBN 978-3-030-36106-8. Citado na página 45.

DODGE, S.; KARAM, L. *Understanding How Image Quality Affects Deep Neural Networks*. arXiv, 2016. Disponível em: <https://arxiv.org/abs/1604.04004>. Citado 2 vezes nas páginas 41 e 44.

ENVISION. *Hear what you want to see*. 2022. Disponível em: <https://www.letsenvision.com/>. Acesso em: 15 mai. 2022. Citado na página 46.

FACEBOOK. *Como funciona o texto alternativo automático no Facebook?* 2022. Disponível em: <https://pt-br.facebook.com/help/216219865403298/?helpref=uf_share>. Acesso em: 15 mai. 2022. Citado na página 45.

FERREIRA, D. d. S. Visão computacional para deificientes visuais: Uma aplicação ios. In: . [s.n.], 2014. Disponível em: <http://ric.cps.sp.gov.br/bitstream/123456789/568/1/20141S_FERREIRADouglasdeSouza_TCCPD1280.pdf>. Acesso em: 15 mai. 2022. Citado na página 45.

FU, C.-Y. et al. *DSSD : Deconvolutional Single Shot Detector*. arXiv, 2017. Disponível em: <https://arxiv.org/abs/1701.06659>. Citado na página 43.

GARCIA, F. E.; NERIS, V. P. de A. Design guidelines for audio games. In: KUROSU, M. (Ed.). *Human-Computer Interaction. Applications and Services*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013. p. 229–238. ISBN 978-3-642-39262-7. Citado na página 28.

GIRSHICK, R. *Fast R-CNN*. arXiv, 2015. Disponível em: <https://arxiv.org/abs/1504.08083>. Citado na página 42.

GIRSHICK, R. et al. *Rich feature hierarchies for accurate object detection and semantic segmentation*. arXiv, 2013. Disponível em: <https://arxiv.org/abs/1311.2524>. Citado na página 42.

GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. *Deep Learning*. [S.l.]: MIT Press, 2016. <http://www.deeplearningbook.org>. Citado 3 vezes nas páginas 34, 35 e 41.

GRANDVIEWRESEARCH. *Voice And Speech Recognition Market Size Report*. 2022. Disponível em: <https://www.grandviewresearch.com/industry-analysis/voice-recognition-market>. Acesso em: 21 jun. 2022. Citado na página 40.

GUO, A. et al. *Toward Fairness in AI for People with Disabilities: A Research Roadmap*. arXiv, 2019. Disponível em: <https://arxiv.org/abs/1907.02227>. Citado 2 vezes nas páginas 38 e 45.

GUTIéRREZ, C. 2010. Disponível em: <https://stackoverflow.com/questions/2174638/whats-the-difference-between-emulation-and-simulation/2175146#2175146>. Acesso em: 14 mai. 2022. Citado na página 24.

HAYKIN, S. *Redes Neurais: Princípios e Prática*. 2. ed. [S.l.]: Bookman Editora, 2001. ISBN 9788577800865. Citado 2 vezes nas páginas 32 e 33.

HE, K. et al. Spatial pyramid pooling in deep convolutional networks for visual recognition. In: *Computer Vision – ECCV 2014*. Springer International Publishing, 2014. p. 346–361. Disponível em: <https://doi.org/10.1007%2F978-3-319-10578-9_23>. Citado na página 42.

HEINEMAN, D. S. *PUBLIC MEMORY AND GAMER IDENTITY: RETROGAMING AS NOSTALGIA*. 2014. Disponível em: <http://gamescriticism.org/articles/heineman-1-1/>. Acesso em: 23 abr. 2022. Citado na página 23.

HIJAZI, S. L.; KUMAR, R.; ROWEN, C. Using convolutional neural networks for image recognition. In: . [s.n.], 2015. Disponível em: <https://ip.cadence.com/uploads/901/cnn_wp-pdf>. Citado 2 vezes nas páginas 37 e 38.

HONDA, H.; FACURE, M.; YAOHAO, P. *Os Três Tipos de Aprendizado de Máquina*. 2017. Disponível em: <https://lamfo-unb.github.io/2017/07/27/tres-tipos-am/>. Acesso em: 21 mai. 2022. Citado na página 31.

IBGE. *População residente por tipo de deficiência permanente, 2010*. 2010. Disponível em: <https://www.ibge.gov.br/estatisticas/sociais/populacao/9662-censo-demografico-2010.html?edicao=9749&t=destaques>. Acesso em: 01 mai. 2022. Citado na página 27.

IBM Corporate. *What are neural networks?* 2021. <https://www.ibm.com/cloud/learn/neural-networks>. [Online; accessed 29-May-2022]. Acesso em: 29 may. 2022. Citado 3 vezes nas páginas 33, 34 e 35.

IGDA. Accessibility in games: Motivations and approaches. *International Game Developers Association*, 6 2004. Citado na página 28.

IGDA. *IGDA GASIG*. 2022. Disponível em: <https://igda.org/sigs/game-accessibility/>. Acesso em: 01 mai. 2022. Citado na página 28.

INTELLIGENCE, D. *Global Video Game Consumer Segmentation*. 2022. Disponível em: <https://www.giiresearch.com/report/dfc1066066-global-video-game-consumer-segmentation.html>. Acesso em: 29 abr. 2022. Citado na página 22.

IOFFE, S.; SZEGEDY, C. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In: BACH, F.; BLEI, D. (Ed.). *Proceedings of the 32nd International Conference on Machine Learning*. Lille, France: PMLR, 2015. (Proceedings of Machine Learning Research, v. 37), p. 448–456. Disponível em: <https://proceedings.mlr.press/v37/ioffe15.html>. Citado na página 36.

JIANG, P. et al. A review of yolo algorithm developments. *Procedia Computer Science*, v. 199, p. 1066–1073, 2022. ISSN 1877-0509. The 8th International Conference on Information Technology and Quantitative Management (ITQM 2020 & 2021): Developing Global Digital Economy after COVID-19. Disponível em: <https://www.sciencedirect.com/science/article/pii/S1877050922001363>. Citado na página 43.

JUANG, B.; RABINER, L. Automatic speech recognition - a brief history of the technology development. In: . [S.l.: s.n.], 2004. Citado 2 vezes nas páginas 19 e 39.

JUNG, S. *Xbox Adaptive Controller Makes Gaming More Accessible For All*. 2018. Disponível em: <https://www.medgadget.com/2018/05/xbox-adaptive-controller-makes-gaming-more-accessible-for-all.html>. Acesso em: 01 mar. 2022. Citado na página 30.

KAVLAKOGLU, E. *AI vs. Machine Learning vs. Deep Learning vs. Neural Networks: What's the Difference?* 2022. <https://www.ibm.com/cloud/blog/ai-vs-machine-learning-vs-deep-learning-vs-neural-networks>. [Online; accessed 29-May-2022]. Acesso em: 29 may. 2022. Citado na página 34.

KRYSCHNACKLONGPLAY. *Pokemon Red Version - Longplay [GB]*. 2021. Disponível em: <https://youtu.be/pvH5hWc7Mes>. Acesso em: 21 ago. 2022. Citado na página 52.

LACERDA, J. *Visão computacional: conheça as aplicações e a importância para a transformação digital*. 2021. Disponível em: <https://certi.org.br/blog/visao-computacional/>. Acesso em: 15 mai. 2022. Citado na página 45.

LECUN, Y. et al. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, v. 86, n. 11, p. 2278–2324, 1998. Citado na página 35.

LEGGETT, J.; WILLIAMS, G. An empirical investigation of voice as an input modality for computer programming. *International Journal of Man-Machine Studies*, v. 21, n. 6, p. 493–520, 1984. ISSN 0020-7373. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0020737384800577>. Citado na página 40.

LEVIS, J.; SUVOROV, R. Automatic speech recognition. In: _____. [S.l.: s.n.], 2012. Citado 2 vezes nas páginas 38 e 39.

LI, F.-F.; JOHNSON, J.; YEUNG, S. *Lecture 11: Detection and Segmentation*. 2017. <http://cs231n.stanford.edu/slides/2017/cs231n_2017_lecture11.pdf>. Acessado: 2022-05-10. Citado na página 41.

LIPKA, P. *GameScripter - A Vision Based Tool for Playing Games*. 2011. <https://www.doc.ic.ac.uk/teaching/distinguished-projects/2011/p.lipka.pdf>. Acesso em: 30 mai. 2022. Citado na página 47.

LIU, W. et al. SSD: Single shot MultiBox detector. In: *Computer Vision – ECCV 2016*. Springer International Publishing, 2016. p. 21–37. Disponível em: <https://doi.org/10.1007%2F978-3-319-46448-0_2>. Citado na página 43.

LONGPLAYSHD. *Longplay - Pokemon Red*. 2019. Disponível em: <https://youtu.be/WjHDuVA5k5A>. Acesso em: 21 ago. 2022. Citado na página 52.

LOWERRE, B. T. The harpy speech recognition system. In: . [S.l.: s.n.], 1976. Citado na página 39.

MACEDO, L. P. *Desenvolvimento de um módulo de reconhecimento de voz para a game engine Godot*. 2017. Monografia final da disciplina, Bacharelado em Ciência da Computação, Universidade de São Paulo. Citado 4 vezes nas páginas 39, 40, 46 e 50.

MAIER, A. *Reinforcement Learning — Part 4*. 2020. <https://towardsdatascience.com/reinforcement-learning-part-4-3c51edd8c4bf>. Acesso em: 30 mai. 2022. Citado na página 31.

MANDAL, M. *Introduction to Convolutional Neural Networks (CNN)*. 2021. Disponível em: <https://www.analyticsvidhya.com/blog/2021/05/convolutional-neural-networks-cnn/>. Acesso em: 21 mai. 2022. Citado 2 vezes nas páginas 35 e 36.

MATHWORKS. *What is a Convolutional Neural Network?* 2022. Disponível em: <https://www.mathworks.com/discovery/convolutional-neural-network-matlab.html>. Acesso em: 05 jun. 2022. Citado na página 36.

MICROSOFT. *PLUGGED IN*. 2018. Disponível em: <https://news.microsoft.com/stories/xbox-adaptive-controller/>. Acesso em: 01 mar. 2022. Citado 2 vezes nas páginas 28 e 29.

NAJIBI, M.; RASTEGARI, M.; DAVIS, L. S. *G-CNN: an Iterative Grid Based Object Detector*. arXiv, 2015. Disponível em: <https://arxiv.org/abs/1512.07729>. Citado na página 43.

NEWMAN, J. *Videogames*. [S.l.]: Routledge, 2004. (Routledge introductions to media and communications). ISBN 9780415281911. Citado na página 23.

NORVIG, P.; RUSSELL, S. *Inteligência artificial: Tradução da 3a Edição*. Elsevier Brasil, 2014. ISBN 9788535251418. Disponível em: <https://www.cin.ufpe.br/~gtsa/Periodo/PDF/4P/SI.pdf>. Citado 2 vezes nas páginas 29 e 30.

NOVAK, J. *Desenvolvimento de games*. [S.l.]: Cengage Learning, 2010. ISBN 8522106320. Citado na página 21.

NOYES, J.; HAIGH, R.; STARR, A. Automatic speech recognition for disabled people. *Applied Ergonomics*, v. 20, n. 4, p. 293–298, 1989. ISSN 0003-6870. Disponível em: <https://www.sciencedirect.com/science/article/pii/0003687089901932>. Citado na página 40.

OPENCV. *About OpenCV*. 2022. Disponível em: <https://opencv.org/about/>. Acesso em: 15 mai. 2022. Citado na página 45.

PAPERT, S. The summer vision project. In: . [S.l.: s.n.], 1966. Citado na página 41.

PATHAK, A. R.; PANDEY, M.; RAUTARAY, S. Application of deep learning for object detection. *Procedia Computer Science*, v. 132, p. 1706–1717, 2018. ISSN 1877-0509. International Conference on Computational Intelligence and Data Science. Disponível em: <https://www.sciencedirect.com/science/article/pii/S1877050918308767>. Citado na página 42.

PEPE, F.; CONTRIBUTORS. *The CRPG book: A Guide to Computer Role-Playing Games*. Digital: www.crpgbook.wordpress.com; Físico: Bitmap Books, 2022. Disponível em: <https://crpgbook.files.wordpress.com/2022/04/crpg-book-expanded-edition_3.2a.pdf>. Citado 2 vezes nas páginas 21 e 22.

PINOLA, M. *Speech Recognition Through the Decades: How We Ended Up With Siri*. 2011. Disponível em: <https://www.pcworld.com/article/477914/speech_recognition_through_the_decades_how_we_ended_up_with_siri.html>. Acesso em: 14 mai. 2022. Citado na página 40.

PLAYSTATION. *The last of us part II - accessibility*. 2020. Disponível em: <https://www.playstation.com/en-us/games/the-last-of-us-part-ii/accessibility/>. Acesso em: 31 mar. 2022. Citado 2 vezes nas páginas 19 e 28.

PULLI, K. et al. Real-time computer vision with opencv. *Commun. ACM*, Association for Computing Machinery, New York, NY, USA, v. 55, n. 6, p. 61–69, jun 2012. ISSN 0001-0782. Disponível em: <https://doi.org/10.1145/2184319.2184337>. Citado na página 45.

RABINER, L.; JUANG, B. *Fundamentals of Speech Recognition*. [S.l.]: PTR Prentice Hall, 1993. (Prentice-Hall Signal Processing Series: Advanced monographs). ISBN 9780130151575. Citado na página 38.

RAGHAVENDRA et al. Deep convolution neural network for accurate diagnosis of glaucoma using digital fundus images. *Information Sciences*, v. 441, 02 2018. Citado 3 vezes nas páginas 35, 37 e 38.

REDMON, J. *Darknet: Open Source Neural Networks in C*. 2013–2016. <http://pjreddie.com/darknet/>. Citado na página 44.

REDMON, J. et al. *You Only Look Once: Unified, Real-Time Object Detection*. arXiv, 2015. Disponível em: <https://arxiv.org/abs/1506.02640>. Citado na página 43.

REDMON, J.; FARHADI, A. *YOLO9000: Better, Faster, Stronger*. arXiv, 2016. Disponível em: <https://arxiv.org/abs/1612.08242>. Citado 2 vezes nas páginas 43 e 45.

REDMON, J.; FARHADI, A. *YOLOv3: An Incremental Improvement*. arXiv, 2018. Disponível em: <https://arxiv.org/abs/1804.02767>. Citado 2 vezes nas páginas 43 e 45.

RIBEIRO, J. Do papel para o digital: A adaptação de contextos educativos digitais de alunos com nee. *Sessões Especiais - Oradores Convidados*, v. 3, p. 9–10, 5 2011. Citado na página 27.

ROSENTHAL, D. S. H. Emulation & virtualization as preservation strategies. In: . [S.l.: s.n.], 2015. Citado 2 vezes nas páginas 19 e 23.

SANT'ANA, D. G. *Controle Por Voz Utilizando a Engine Julius Com Fala Contínua*. 2014. <http://dsc.inf.furb.br/arquivos/tccs/monografias/2014_1_deivid-geovani-santana_monografia.pdf>. Acesso em: 30 mai. 2022. Citado 2 vezes nas páginas 46 e 50.

SASSAKI, R. K. Inclusão: acessibilidade no lazer, trabalho e educação. *Revista Nacional de Reabilitação (Reação)*, p. 10–16, 3 2009. Citado na página 26.

SERMANET, P. et al. *OverFeat: Integrated Recognition, Localization and Detection using Convolutional Networks*. arXiv, 2013. Disponível em: <https://arxiv.org/abs/1312.6229>. Citado na página 43.

SHARMA, A. An incremental improvement with darknet-53 and multi-scale predictions (YOLOv3). In: CHAKRABORTY, D. et al. (Ed.). *PyImageSearch*. [S.l.: s.n.], 2022. Https://pyimg.co/8xfpg. Citado na página 45.

SHEN, Z. et al. *DSOD: Learning Deeply Supervised Object Detectors from Scratch*. arXiv, 2017. Disponível em: <https://arxiv.org/abs/1708.01241>. Citado na página 43.

SHINDE, P. P.; SHAH, S. A review of machine learning and deep learning applications. In: *2018 Fourth International Conference on Computing Communication Control and Automation (ICCUBEA)*. [S.l.: s.n.], 2018. p. 1–6. Citado na página 31.

SWITCH, O. *Game Not Over*. 2022. Disponível em: <https://oneswitch.org.uk/page/switch-gaming>. Acesso em: 01 mai. 2022. Citado na página 28.

SZEGEDY, C.; TOSHEV, A.; ERHAN, D. Deep neural networks for object detection. In: BURGES, C. et al. (Ed.). *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2013. v. 26. Disponível em: <https://proceedings.neurips.cc/paper/2013/file/f7cade80b7cc92b991cf4d2806d6bd78-Paper.pdf>. Citado na página 43.

TAKHTEYEV, Y.; DUPONT, Q. Retrocomputing as preservation and remix. *Library Hi Tech*, v. 31, p. 355–370, 06 2013. Citado na página 23.

TANENBAUM, A. S. *Organização estruturada de computadores/*. 6. ed.. ed. São Paulo:: Pearson Prentice Hall,, 2013. Citado 2 vezes nas páginas 24 e 25.

TIBCO. *O que é aprendizagem não supervisionada?* 2022. Disponível em: <https://www.tibco.com/pt-br/reference-center/what-is-unsupervised-learning#:~:text=O%20aprendizado%20n%C3%A3o%20supervisionado%20%C3%A9,tentar%20entender%20por%20conta%20pr%C3%B3pria.> Acesso em: 21 mai. 2022. Citado na página 31.

TONON, E. G. Modelo de visão computacional para auxílio a deficientes visuais na locomoção urbana. In: . [s.n.], 2021. Disponível em: <https://repositorio.ucs.br/xmlui/handle/11338/9680?locale-attribute=pt_BR>. Acesso em: 15 mai. 2022. Citado na página 45.

TORRALBA et al. Context-based vision system for place and object recognition. In: *Proceedings Ninth IEEE International Conference on Computer Vision*. [S.l.: s.n.], 2003. p. 273–280 vol.1. Citado na página 41.

TORRES, E. F.; MAZZONI, A. A. Conteúdos digitais multimídia: o foco na usabilidade e acessibilidade. *Ciência da Informação*, v. 33, p. 152–160, 3 2004. Citado na página 27.

TSANG, S.-H. Review — cspnet: A new backbone that can enhance learning capability of cnn. In: . [S.l.: s.n.], 2021. Https://sh-tsang.medium.com/review-cspnet-a-new-backbone-that-can-enhance-learning-capability-of-cnn-da7ca51524bf. Citado na página 45.

UFJF, G. d. T. s. A. *Deficiência Físico-motora (DFM)*. 2022. Disponível em: <https://www2.ufjf.br/gtacessibilidadeufjfgv/deficiencia-fisico-motora>. Acesso em: 01 mar. 2022. Citado na página 28.

ULSON, L. M. e J. A. A aplicaÇÃo de redes neurais profundas para detecÇÃo e classificaÇÃo de plantas daninhas e seu estado da arte. *REGRAD - Revista Eletrônica de Graduação do UNIVEM - ISSN 1984-7866*, v. 11, n. 01, p. 391–403, 2018. ISSN 1984-7866. Disponível em: <https://revista.univem.edu.br/REGRAD/article/view/2638>. Citado na página 41.

USSELMAN, S. W. Building ibm: Shaping an industry and its technology. by emerson w. pugh · cambridge, mass.: Mit press, 1995. xvi 403 pp. illustrations, tables, charts, notes, and index. $29.95. isbn 0-262-16147-8. *Business History Review*, Cambridge University Press, v. 69, n. 3, p. 443–445, 1995. Citado na página 23.

WALD, M. Enhancing accessibility through automatic speech recognition. *Proceedings of Accessible Design in the Digital World*, 2005. Event Dates: 23-25 august 2005. Disponível em: <https://eprints.soton.ac.uk/262143/>. Citado na página 40.

WANGENHEIM, A. von. *Visão Computacional::Deep Learning*. 2018. <https://lapix.ufsc.br/ensino/visao/visao-computacionaldeep-learning/>. Acessado: 2022-05-10. Citado na página 42.

WEBAIM. *Motor Disabilities*. 2020. Disponível em: <https://webaim.org/articles/motor/>. Acesso em: 01 mar. 2022. Citado 2 vezes nas páginas 28 e 29.

WIJMAN, T. *Newzoo Global Games Market Report 2021 | Free Version*. 2021. Disponível em: <https://newzoo.com/insights/trend-reports/newzoo-global-games-market-report-2021-free-version>. Acesso em: 28 mai. 2022. Citado 2 vezes nas páginas 19 e 22.

ZHAO, Z.-Q. et al. *Object Detection with Deep Learning: A Review*. arXiv, 2018. Disponível em: <https://arxiv.org/abs/1807.05511>. Citado na página 42.

ZHU, X. et al. *TPH-YOLOv5: Improved YOLOv5 Based on Transformer Prediction Head for Object Detection on Drone-captured Scenarios*. arXiv, 2021. Disponível em: <https://arxiv.org/abs/2108.11539>. Citado na página 43.